UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FCFM

Análisis Numérico

M.A. María Del Carmen Martínez Cejudo

PRODUCTO INTEGRADOR DE APRENDIZAJE

Sebastián Calderón Carrillo 2087472

Edson Armando Amaya Macias 2151976

Juan José Trujillo Garza 1972407

Grupo 031

Contenido

Introducción 3
Codificación 3
Secuencia del cálculo 4
Metodo montante 5
Correr 8
Resolver Sistema 10
Resolver Coeficientes 11
Resoluble 12
Regla del Trapecio 14
Rango 15
Pausar 17
Obtener Numero 18
Mostrar Resultados 19
Mostrar Matriz 22
Leer Opción 23
Leer Numero de Trapecios 24
Leer Fracción 25
Hallar Pivote 26
Formatear Polinomio 27
Formatear Fracción 29
Formatear Decimal 30
Eliminación Montante 31
Ejecutar Cálculo 32
Determinante 33
Construcción del Sistema 34
Conclusión general 35
Ejemplo de ejecución 36
Presentacion 38

Introducción

Los centros de datos requieren un suministro eléctrico continuo para mantener en funcionamiento sus servidores y equipos. Para reducir el riesgo de interrupciones, cuentan con baterías de respaldo que almacenan energía cuando existe un excedente y la entregan cuando se necesita. Por ello, conocer el comportamiento de la potencia durante el día permite estimar si las baterías terminaron cargándose o descargándose.

Nuestro proyecto desarrolla una aplicación de consola en Python que calcula el balance neto de energía de las baterías a partir de mediciones de potencia tomadas en distintas horas. El programa construye un polinomio que interpola los datos ingresados mediante un sistema de Vandermonde, resuelto con el método de Montante, y posteriormente integra la función obtenida en el intervalo de 0 a 24 horas mediante la regla compuesta del trapecio.

La potencia positiva representa energía que entra al sistema y carga las baterías; la potencia negativa representa energía que sale y provoca su descarga. El resultado se expresa en kilowatt-hora (kWh) y representa el cambio neto respecto a una carga inicial que no se conoce.

Codificación

El programa está organizado en módulos para separar la captura de datos, los métodos numéricos y la presentación de resultados. El punto de entrada es **src/main.py**, que importa y ejecuta la función **run()** del módulo **menu.py**.

Descripción de los módulos principales

- **main.py**: inicia la aplicación llamando al menú principal.
- **menu.py**: muestra las opciones, solicita las mediciones y el número de trapecios, construye el sistema, ejecuta los cálculos y presenta la interpretación del balance.
- **ajuste.py**: construye la matriz de Vandermonde, obtiene los coeficientes del polinomio y permite evaluarlo.
- **montante.py**: implementa la eliminación de Montante para resolver el sistema de ecuaciones y contiene funciones auxiliares para matrices.
- **integracion.py**: implementa la regla compuesta del trapecio para aproximar la integral.
- **utils.py**: contiene funciones para leer y validar entradas, convertir números a fracciones, dar formato a los resultados y controlar la consola.

Secuencia del cálculo

Para conservar exactitud durante las operaciones numéricas, el código utiliza la clase Fraction de Python. Así, los coeficientes y la integral se manejan como fracciones exactas y se muestran también en forma decimal aproximada.

Ejecucion de menú

```python
"""Menú interactivo para la administración de baterías de un centro de datos."""

from fractions import Fraction

from modules.ajuste import (
    construir_sistema,
    evaluar_polinomio,
    formatear_polinomio,
    resolver_coeficientes,
)
from modules.integracion import regla_del_trapecio
from modules.montante import mostrar_matriz
from modules.utils import (
    borrar_consola,
    formatear_a_decimal,
    formatear_a_fraccion,
    leer_fraccion,
    leer_int,
    leer_opcion,
    pausar,
)

HORA_INICIO = Fraction(0)
HORA_FIN = Fraction(24)


def _ingresar_mediciones():···


def _leer_numero_trapecios():···


def _mostrar_resultados(puntos, coeficientes, numero_trapecios, energia):···


def ejecutar_calculo():···


def run():···
```

Metodo montante

```python
from fractions import Fraction

from modules.utils import formatear_a_fraccion


def mostrar_matriz(matriz: list = [], titulo: str = "Matriz"):
    if not matriz:
        print("  (Matriz vacía)")
        return
    filas = [[formatear_a_fraccion(valor) for valor in fila] for fila in matriz]
    columnas = len(filas[0])
    anchos = [
        max(len(filas[i][j]) for i in range(len(filas))) for j in range(columnas)
    ]
    print(f"--- {titulo} ---")
    for fila in filas:
        print("  " + "  ".join(cell.rjust(anchos[j]) for j, cell in enumerate(fila)))


def _copiar(matriz):
    return [fila[:] for fila in matriz]


def _hallar_pivote(matriz, k: int) -> bool:
    """Busca un pivote válido empezando en fila y columna k.

    Cambia filas si es necesario.

    Retorna False si toda la columna es 0 (matriz singular).
    """
    if matriz[k][k] != 0:
        return True
    for fila in range(k + 1, len(matriz)):
        if matriz[fila][k] != 0:
            matriz[k], matriz[fila] = matriz[fila], matriz[k]
            return True
    return False

```

```python
def eliminacion_montante(aumentada):
    """Aplica el método de eliminación de Montante a la matriz aumentada.

    ValueError si algún pivote es 0 (sistema singular).
    """
    matriz = _copiar(aumentada)
    n = len(matriz)
    columnas = len(matriz[0]) if n else 0
    pivote_anterior = Fraction(1)
    for k in range(n):
        if not _hallar_pivote(matriz, k):
            raise ValueError(
                "El sistema es singular: pivote igual a cero en la etapa " f"{k + 1}"
            )
        pivote = matriz[k][k]
        for i in range(n):
            if i == k:
                continue
            for j in range(columnas):
                if j == k:
                    continue
                matriz[i][j] = (
                    pivote * matriz[i][j] - matriz[i][k] * matriz[k][j]
                ) / pivote_anterior
        pivote_anterior = pivote
    return matriz


```

```python
def determinante(matriz):
    """Calcula el determinante usando el método de Montante."""
    n = len(matriz)
    if n == 0:
        return Fraction(0)
    matriz_temp = _copiar(matriz)
    signo = 1
    pivote_anterior = Fraction(1)
    for k in range(n - 1):
        if matriz_temp[k][k] == 0:
            matriz_temp_cambiada = _hallar_pivote(matriz_temp, k)
            if not matriz_temp_cambiada:
                return Fraction(0)
            signo = -signo
        pivote = matriz_temp[k][k]
        for i in range(k + 1, n):
            for j in range(k + 1, n):
                matriz_temp[i][j] = (
                    pivote * matriz_temp[i][j]
                    - matriz_temp[i][k] * matriz_temp[k][j]
                ) / pivote_anterior
        pivote_anterior = pivote
    return signo * matriz_temp[n - 1][n - 1]


def resolver_sistema(coeficientes, constantes):
    """Resuelve el sistema de ecuaciones con Montante.

    Retorna una lista de valores para cada incógnita.
    ValueError si no tiene solución única.
    """
    n = len(coeficientes)
    aumentada = [fila + [b] for fila, b in zip(coeficientes, constantes)]
    reducida = eliminacion_montante(aumentada)
    ultimo_pivote = reducida[n - 1][n - 1]
    if ultimo_pivote == 0:
        raise ValueError(
            "El sistema no tiene solución única (determinante igual a cero)"
        )
    return [reducida[i][n] / ultimo_pivote for i in range(n)]
```

```python


def rango(matriz):
    """Auxiliar para saber qué tipo es una matriz con determinante 0."""
    matriz_temp = _copiar(matriz)
    n = len(matriz_temp)
    columnas = len(matriz_temp[0]) if n else 0
    fila_actual = 0
    for col in range(columnas):
        pivote_fila = None
        for i in range(fila_actual, n):
            if matriz_temp[i][col] != 0:
                pivote_fila = i
                break
        if pivote_fila is None:
            continue
        matriz_temp[fila_actual], matriz_temp[pivote_fila] = (
            matriz_temp[pivote_fila],
            matriz_temp[fila_actual],
        )
        pivote = matriz_temp[fila_actual][col]
        for j in range(col, columnas):
            matriz_temp[fila_actual][j] /= pivote
        for i in range(n):
            if i != fila_actual and matriz_temp[i][col] != 0:
                factor = matriz_temp[i][col]
                for j in range(col, columnas):
                    matriz_temp[i][j] -= factor * matriz_temp[fila_actual][j]
        fila_actual += 1
        if fila_actual == n:
            break
    return fila_actual


```

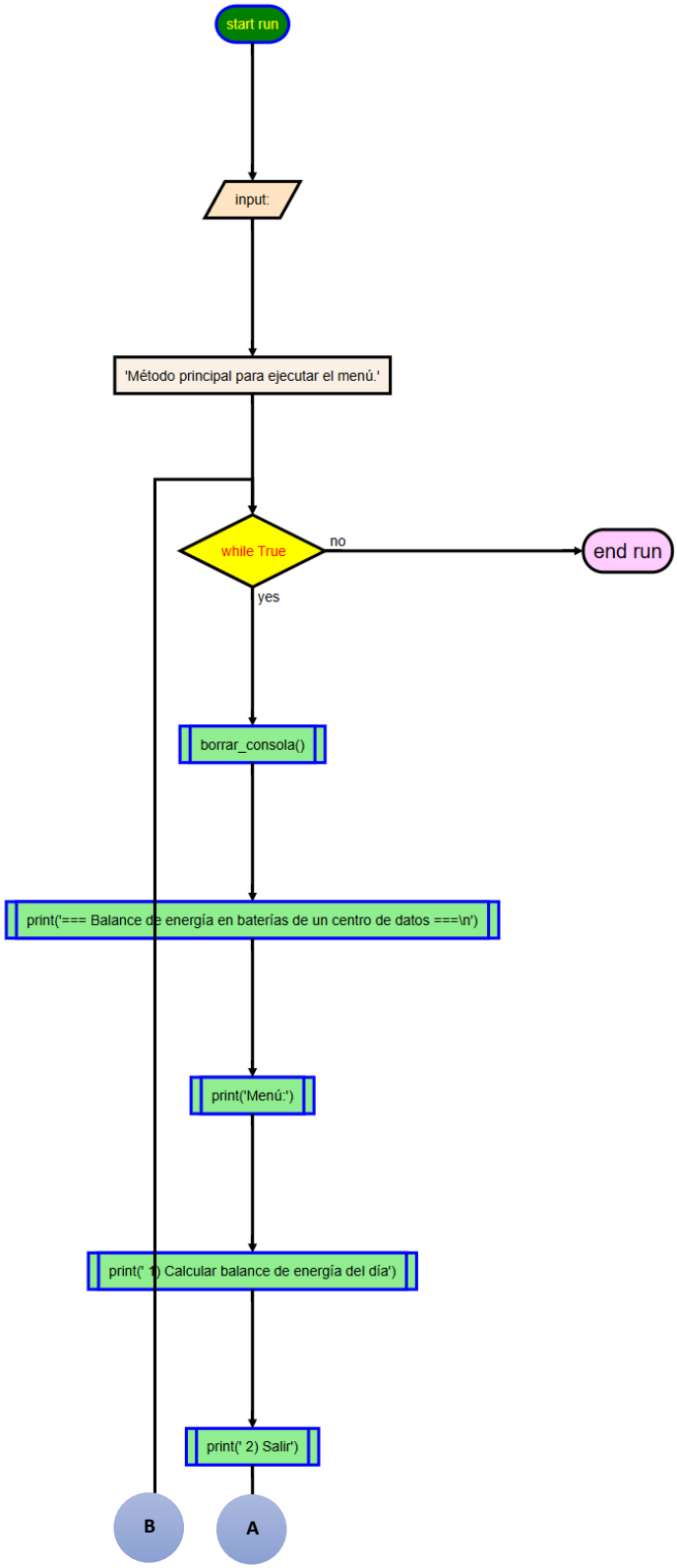
```python

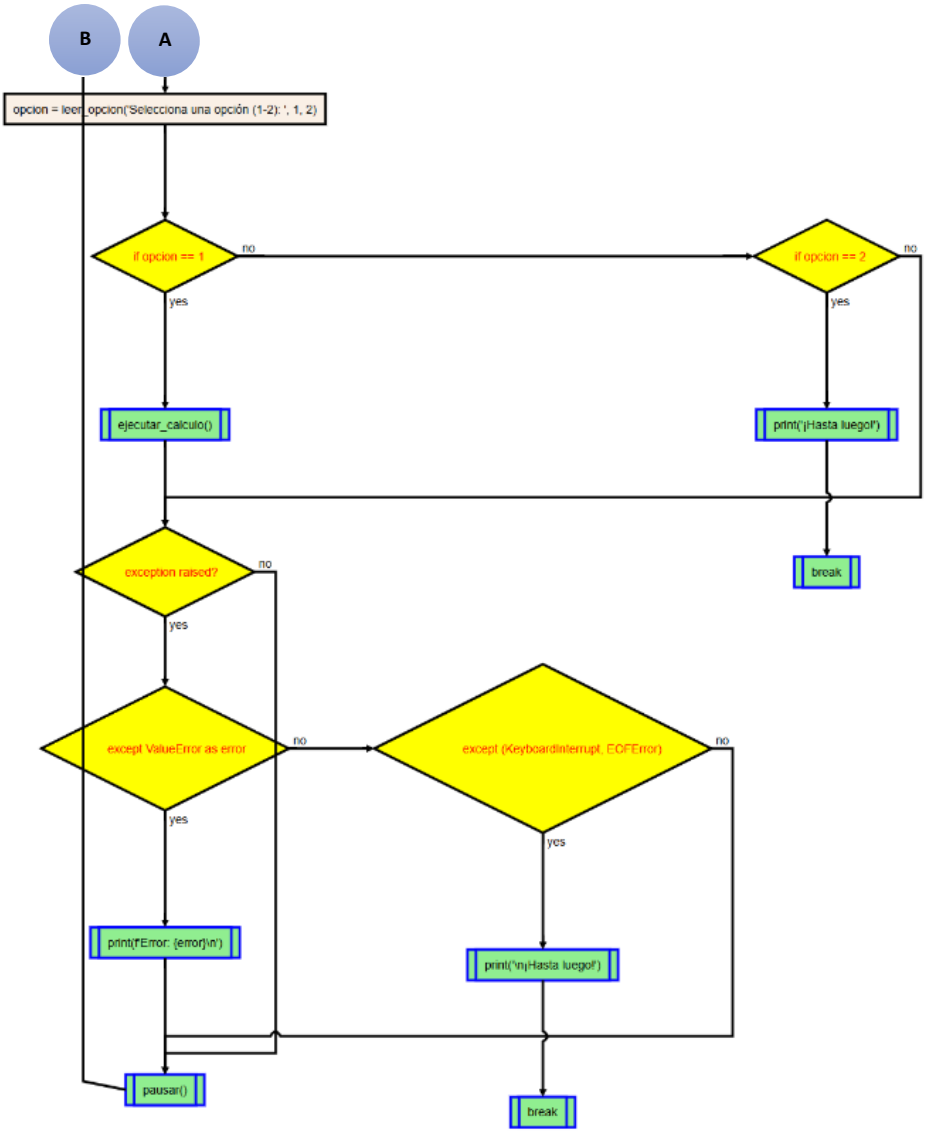
def resoluble(coeficientes, constantes):
    """Verifica si el sistema es resoluble.

    Retorna una tupla (es_resoluble, descripción).
    """
    aumentada = [fila + [b] for fila, b in zip(coeficientes, constantes)]
    if determinante(coeficientes) != 0:
        return True, (
            "Sistema compatible determinado: solución única "
            "(determinante distinto de cero)."
        )
    rango_a = rango(coeficientes)
    rango_ab = rango(aumentada)
    if rango_a == rango_ab:
        return True, (
            "Sistema compatible indeterminado: tiene infinitas soluciones "
            "(determinante cero, rangos iguales)."
        )
    return False, (
        "Sistema incompatible: no tiene solución "
        "(determinante cero y rangos distintos)."
    )

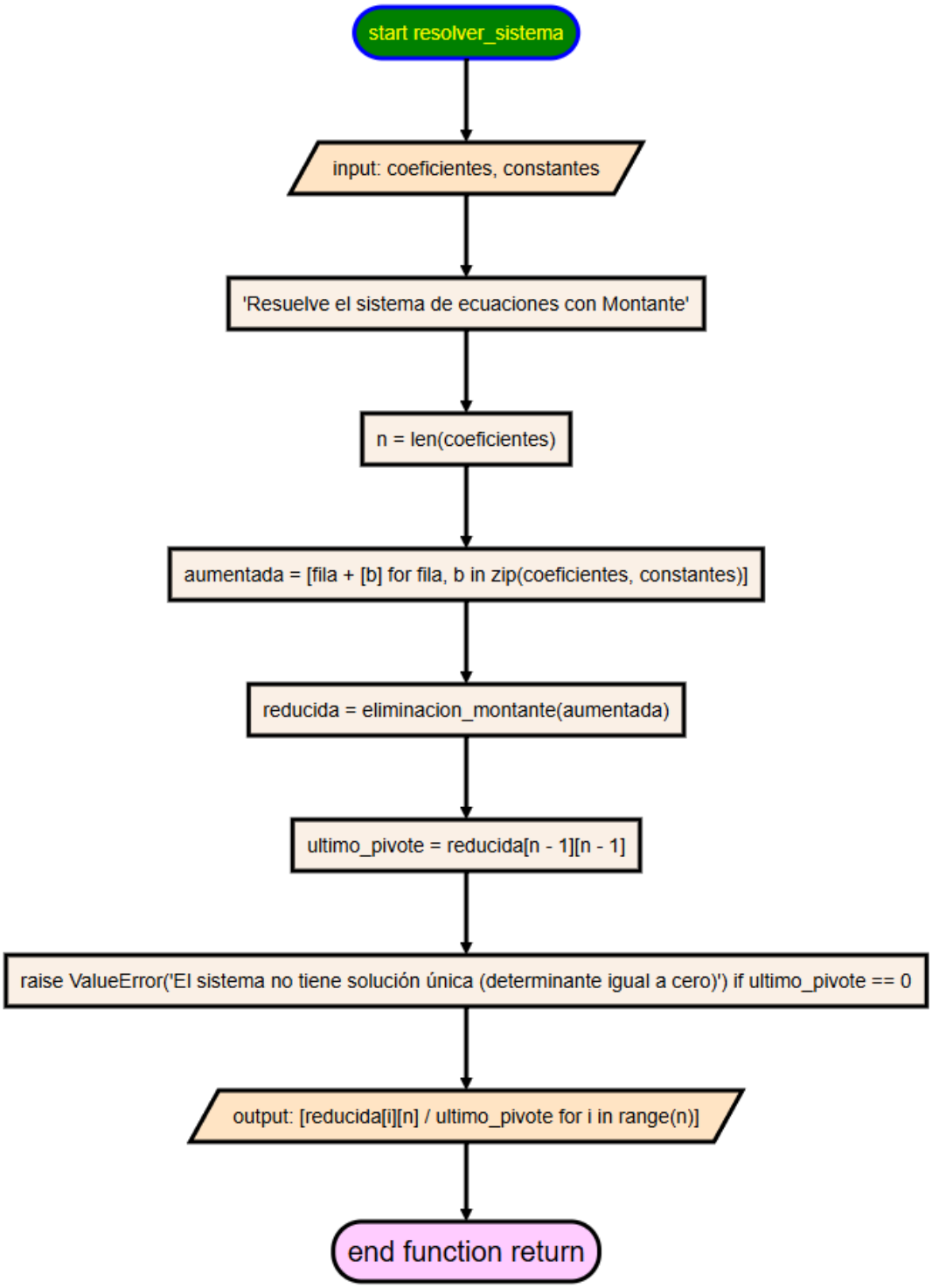
```

Correr

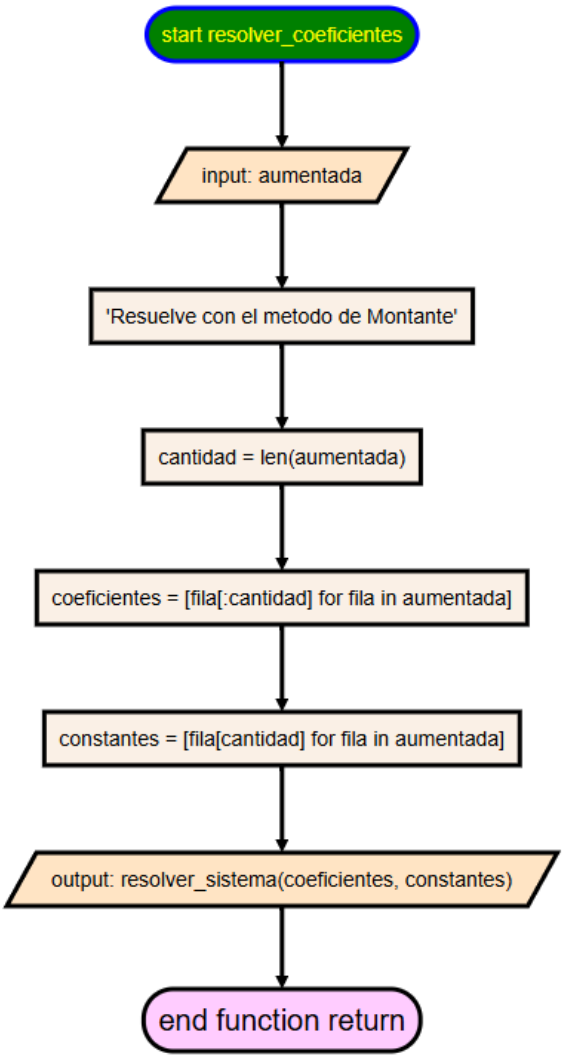

start run
input:
'Método principal para ejecutar el menú.'
while True
no
end run
yes
borrar_consola()
print('=== Balance de energía en baterías de un centro de datos ===\n')
print('Menú:')
print(' 1) Calcular balance de energía del día')
print(' 2) Salir')
B
A


B
A
opcion = leer_opcion('Selecciona una opción (1-2): ', 1, 2)
if opcion == 1
no
yes
if opcion == 2
no
yes
ejecutar_calculo()
print('¡Hasta luego!')
break
exception raised?
no
yes
except ValueError as error
no
except (KeyboardInterrupt, EOFError)
no
yes
yes
print(f'Error: {error}\n')
print('\n¡Hasta luego!')
pausar()
break

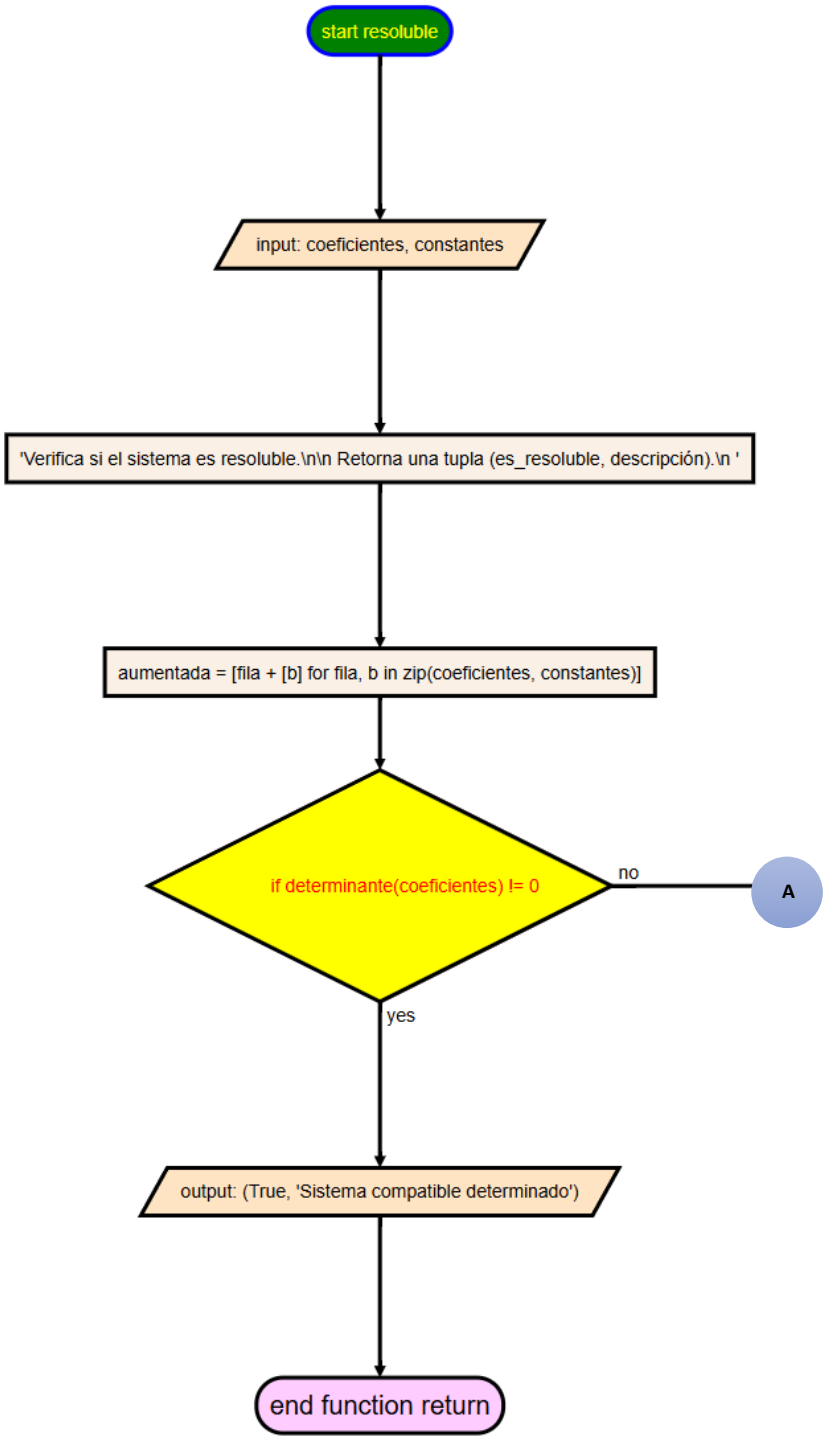
Resolver Sistema

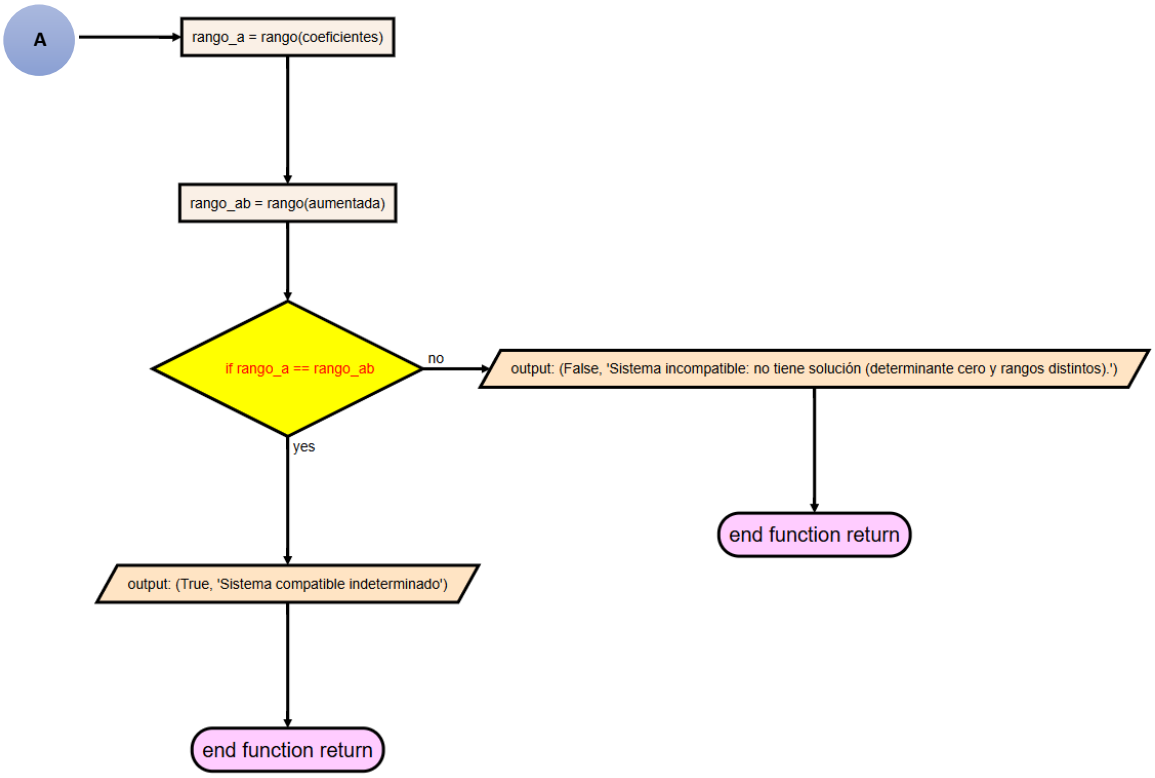


Resolver Coeficientes

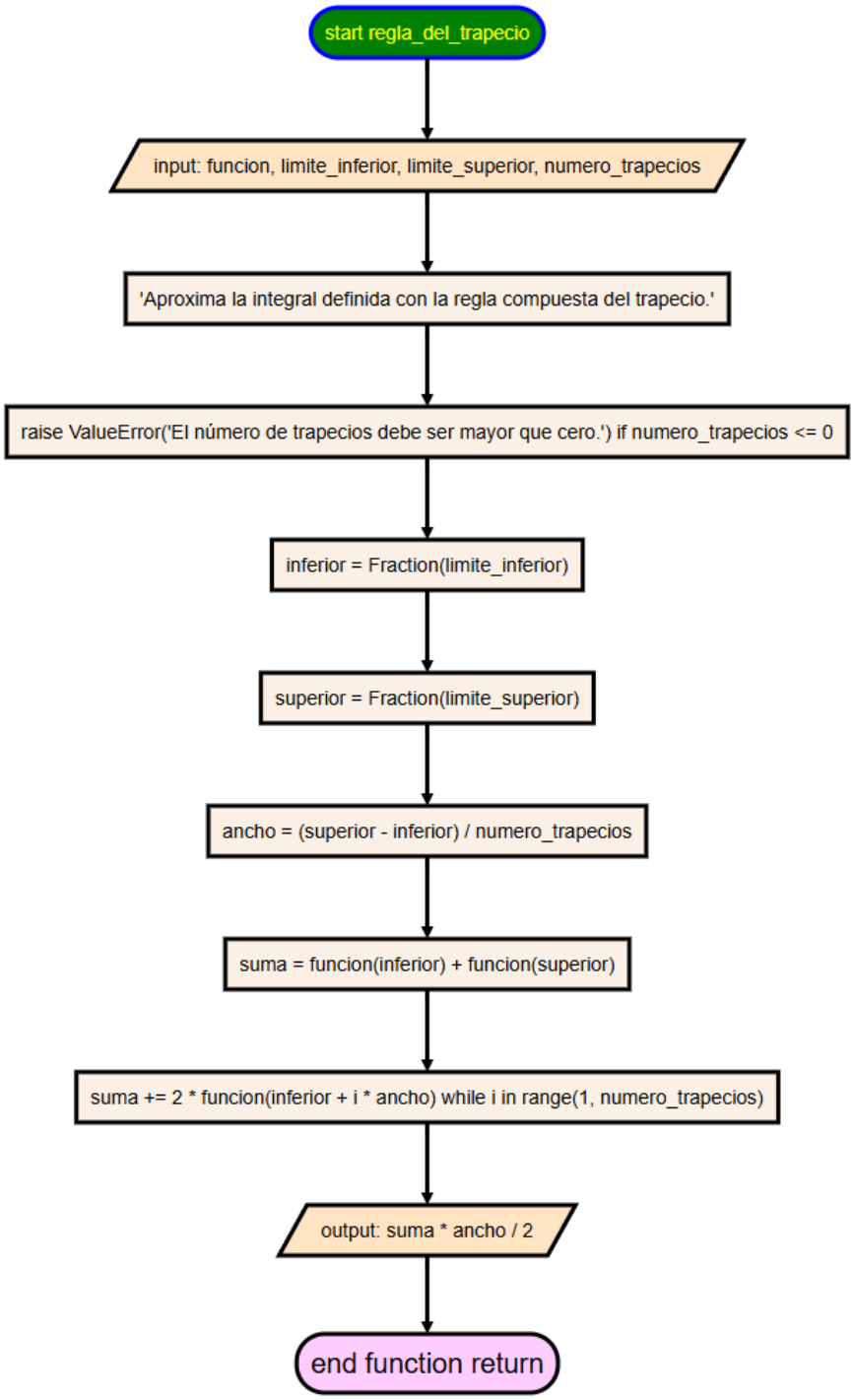


Resoluble

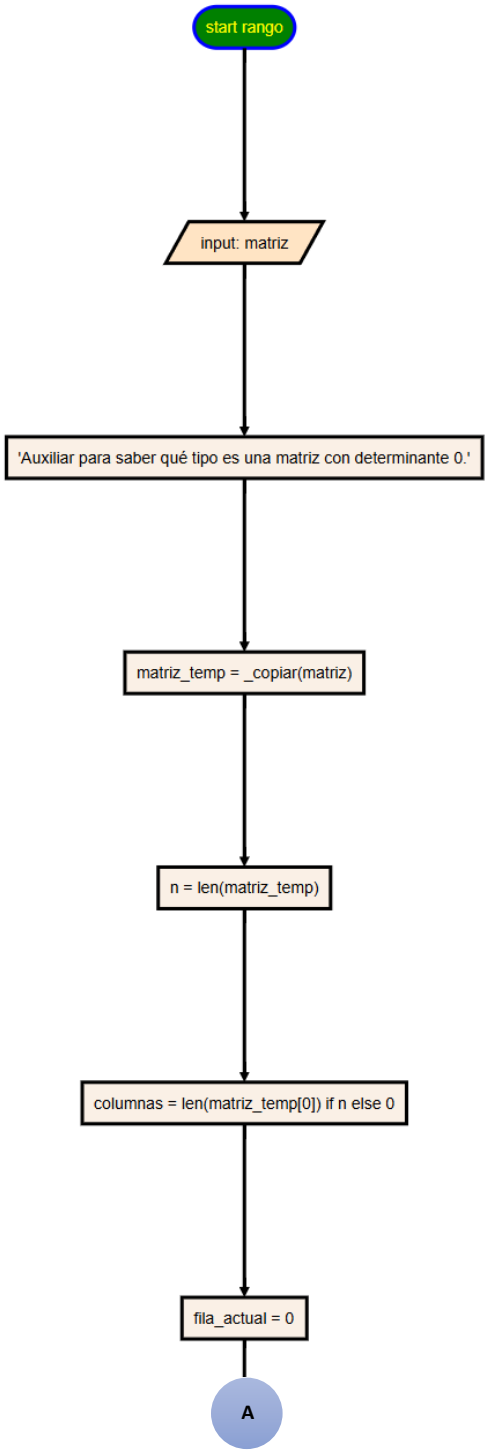

start resoluble
input: coeficientes, constantes
'Verifica si el sistema es resoluble.\n\n Retorna una tupla (es_resoluble, descripción).\n '
aumentada = [fila + [b] for fila, b in zip(coeficientes, constantes)]
if determinante(coeficientes) != 0
no
A
yes
output: (True, 'Sistema compatible determinado')
end function return

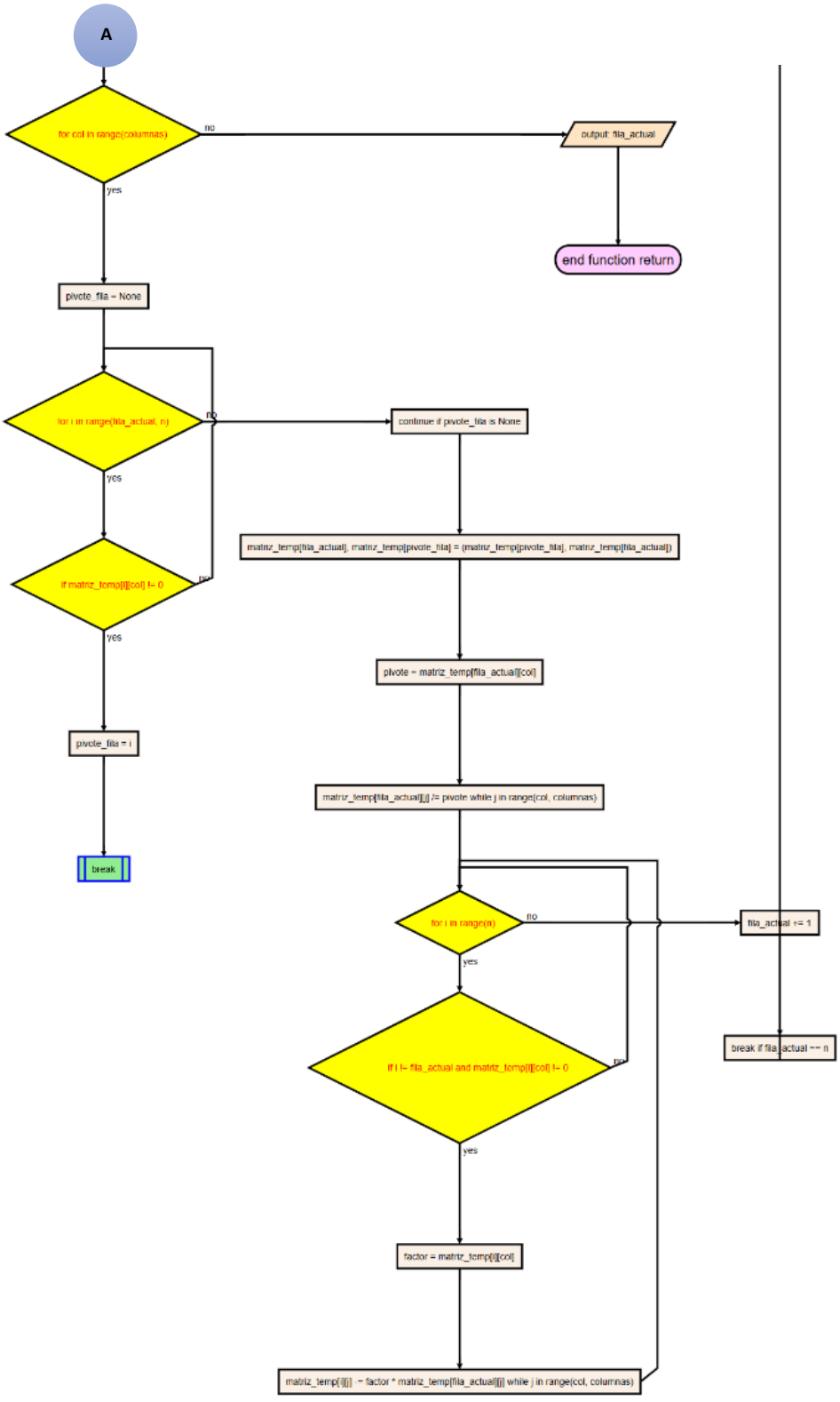

A
rango_a = rango(coeficientes)
rango_ab = rango(aumentada)
if rango_a == rango_ab
no
output: (False, 'Sistema incompatible: no tiene solución (determinante cero y rangos distintos).')
end function return
yes
output: (True, 'Sistema compatible indeterminado')
end function return

Regla del Trapecio

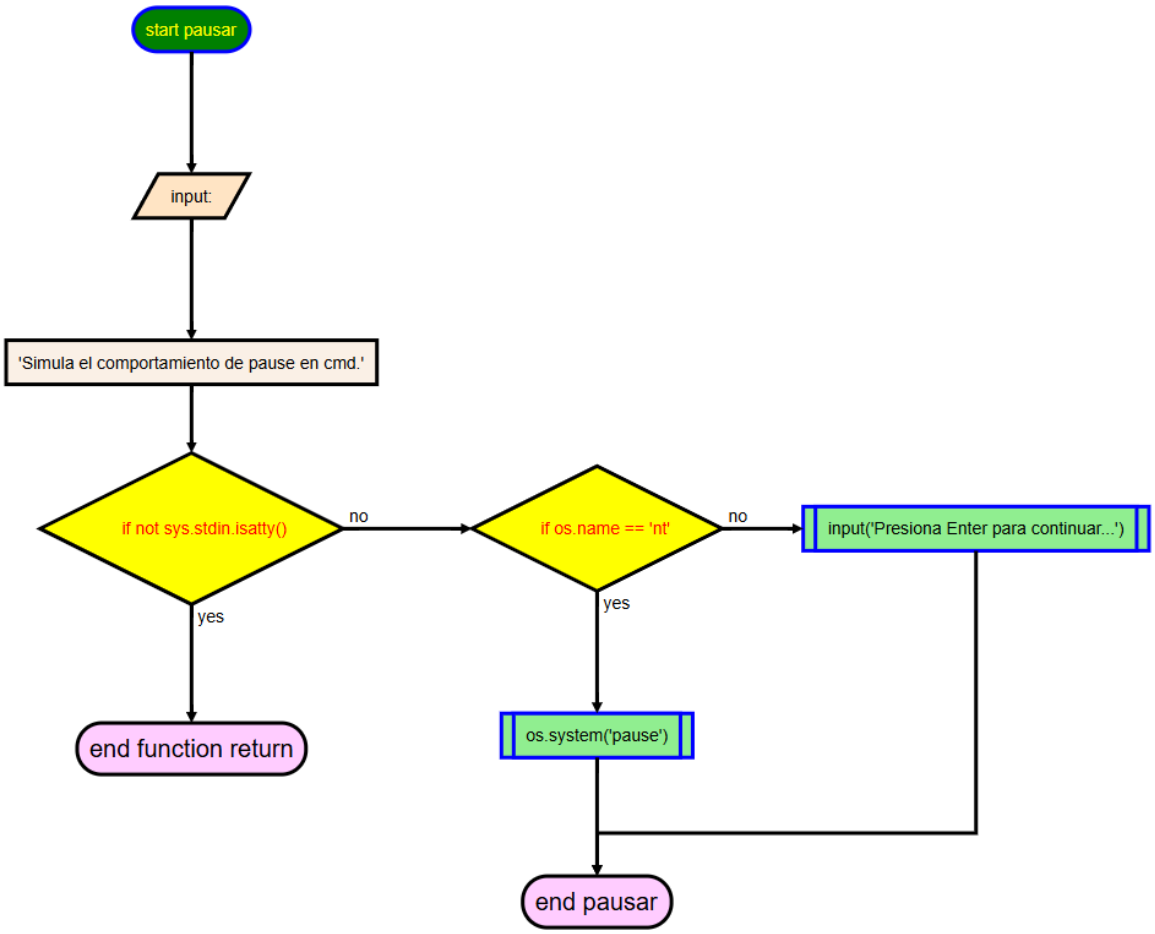


Rango

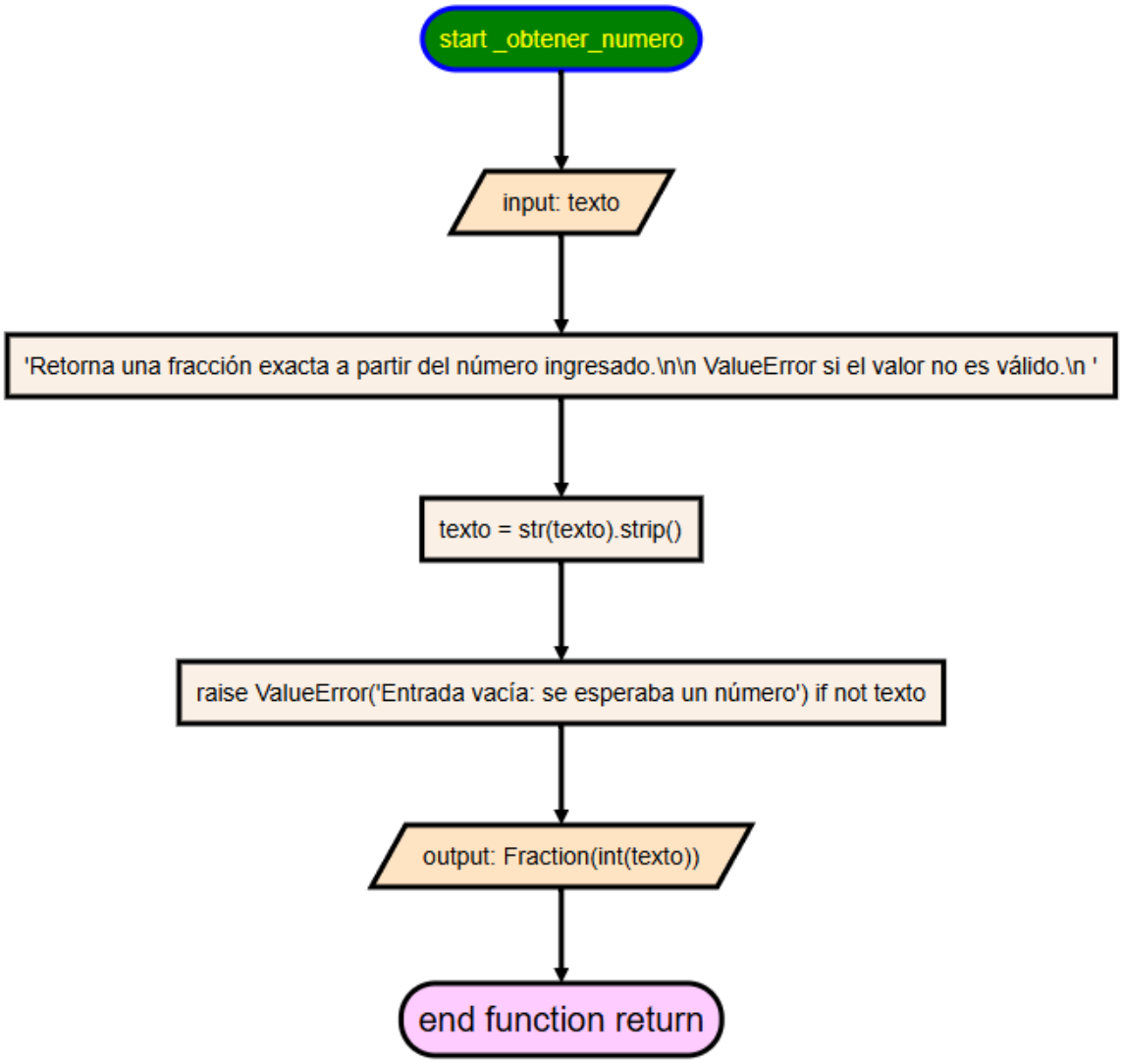

start rango
input: matriz
'Auxiliar para saber qué tipo es una matriz con determinante 0.'
matriz_temp = _copiar(matriz)
n = len(matriz_temp)
columnas = len(matriz_temp[0]) if n else 0
fila_actual = 0
A


A
for col in range(columnas)
no
output: fila_actual
end function return
yes
pivote_fila = None
for i in range(fila_actual, n)
no
continue if pivote_fila is None
yes
matriz_temp[fila_actual], matriz_temp[pivote_fila] = (matriz_temp[pivote_fila], matriz_temp[fila_actual])
if matriz_temp[i][col] != 0
no
yes
pivote = matriz_temp[fila_actual][col]
pivote_fila = i
matriz_temp[fila_actual][j] /= pivote while j in range(col, columnas)
break
for i in range(n)
no
fila_actual += 1
yes
break if fila_actual == n
if i != fila_actual and matriz_temp[i][col] != 0
no
yes
factor = matriz_temp[i][col]
matriz_temp[i][j] -= factor * matriz_temp[fila_actual][j] while j in range(col, columnas)

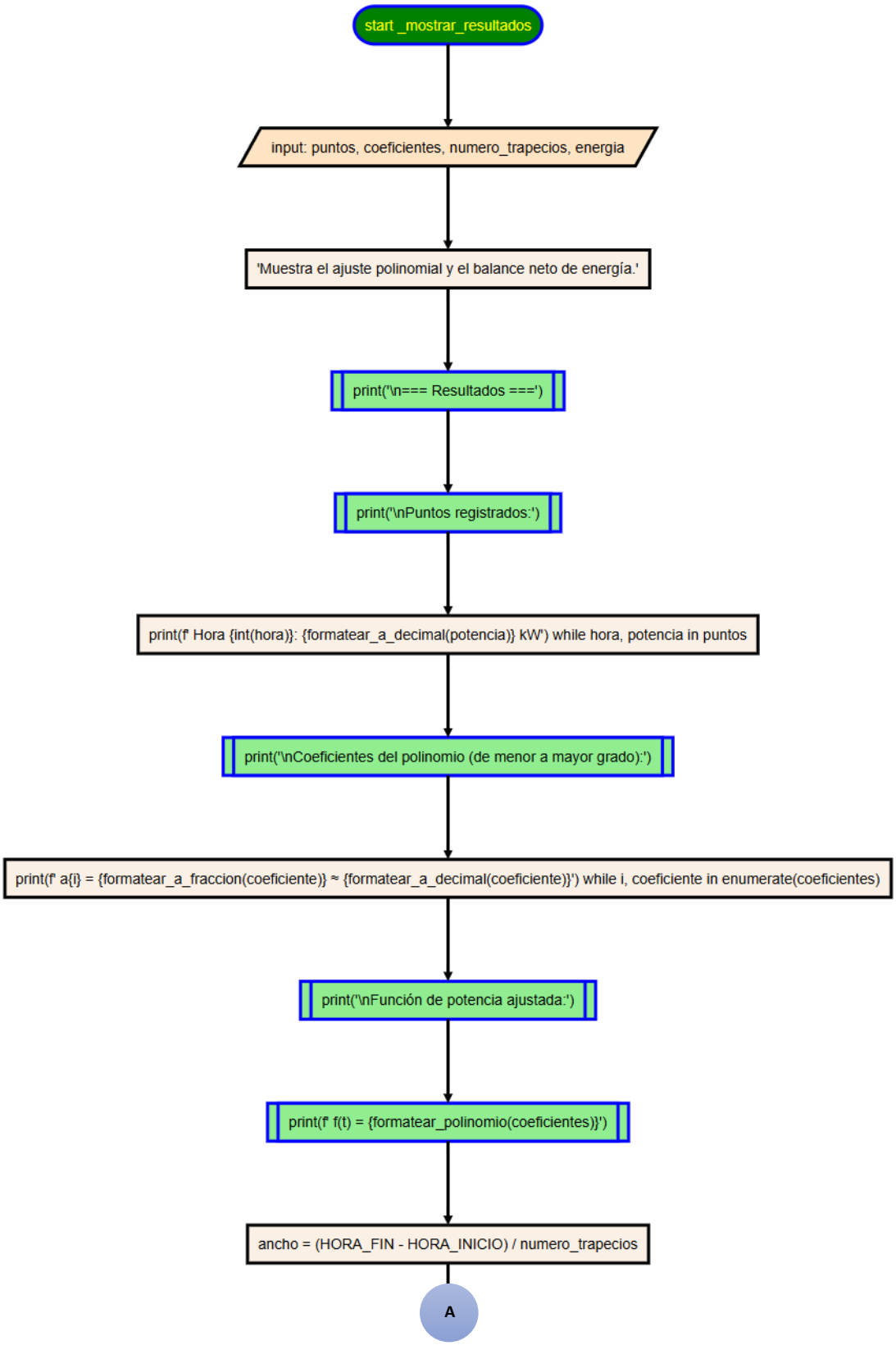
Pausar

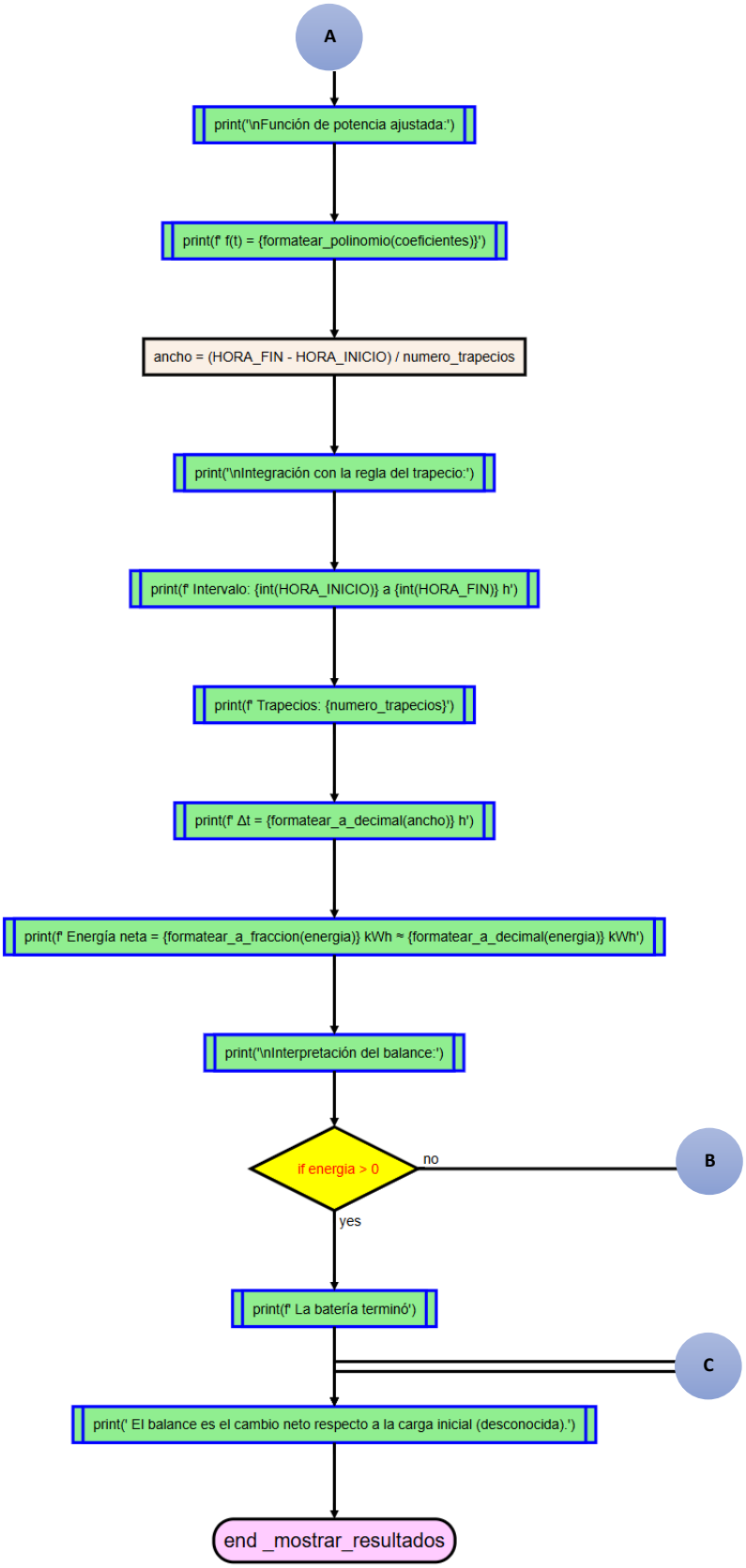

start pausar
input:
'Simula el comportamiento de pause en cmd.'
if not sys.stdin.isatty()
no
if os.name == 'nt'
no
input('Presiona Enter para continuar...')
yes
yes
end function return
os.system('pause')
end pausar

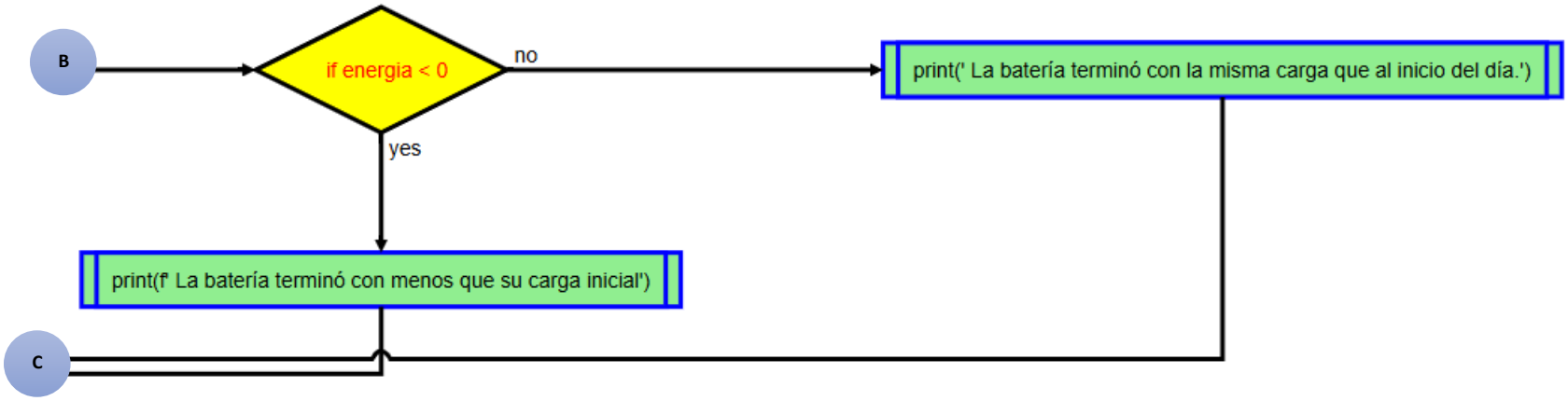
Obtener Numero



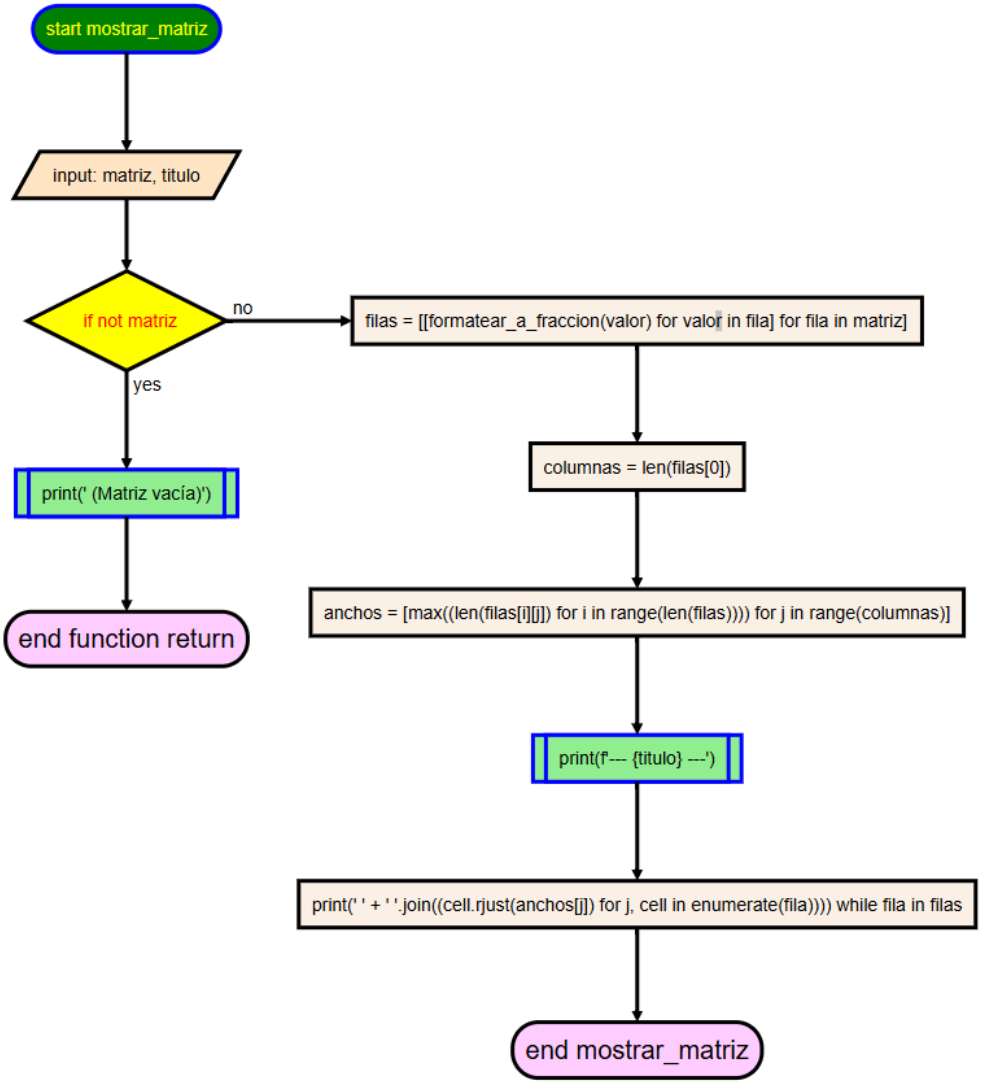
Mostrar Resultados



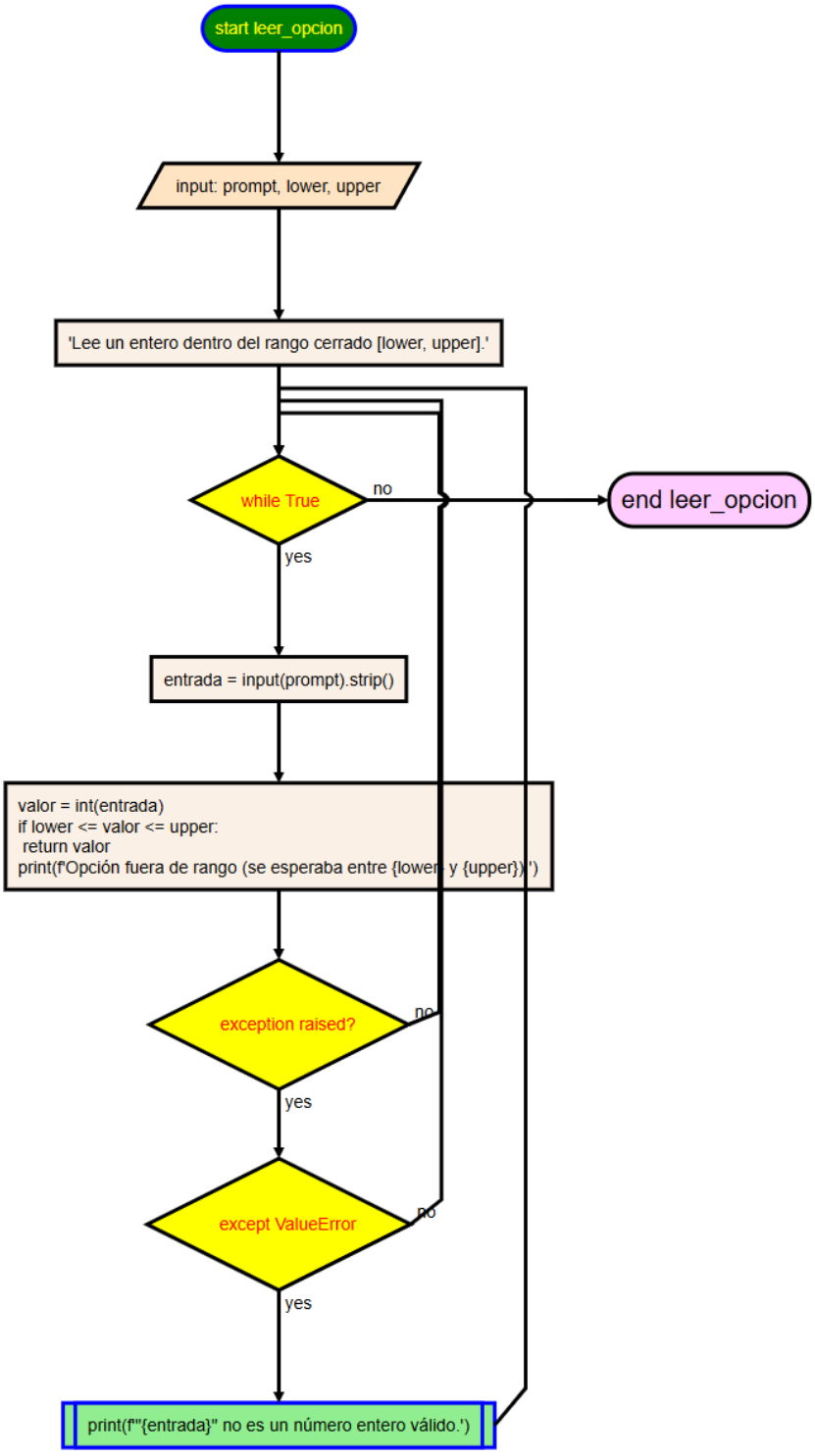

A
print('\nFunción de potencia ajustada:')
print(f' f(t) = {formatear_polinomio(coeficientes)}')
ancho = (HORA_FIN - HORA_INICIO) / numero_trapecios
print('\nIntegración con la regla del trapecio:')
print(f' Intervalo: {int(HORA_INICIO)} a {int(HORA_FIN)} h')
print(f' Trapecios: {numero_trapecios}')
print(f' Δt = {formatear_a_decimal(ancho)} h')
print(f' Energía neta = {formatear_a_fraccion(energia)} kWh ≈ {formatear_a_decimal(energia)} kWh')
print('\nInterpretación del balance:')
if energia > 0
no
B
yes
print(f' La batería terminó')
C
print(' El balance es el cambio neto respecto a la carga inicial (desconocida).')
end _mostrar_resultados


B
if energia < 0
no
print(' La batería terminó con la misma carga que al inicio del día.')
yes
print(f' La batería terminó con menos que su carga inicial')
C

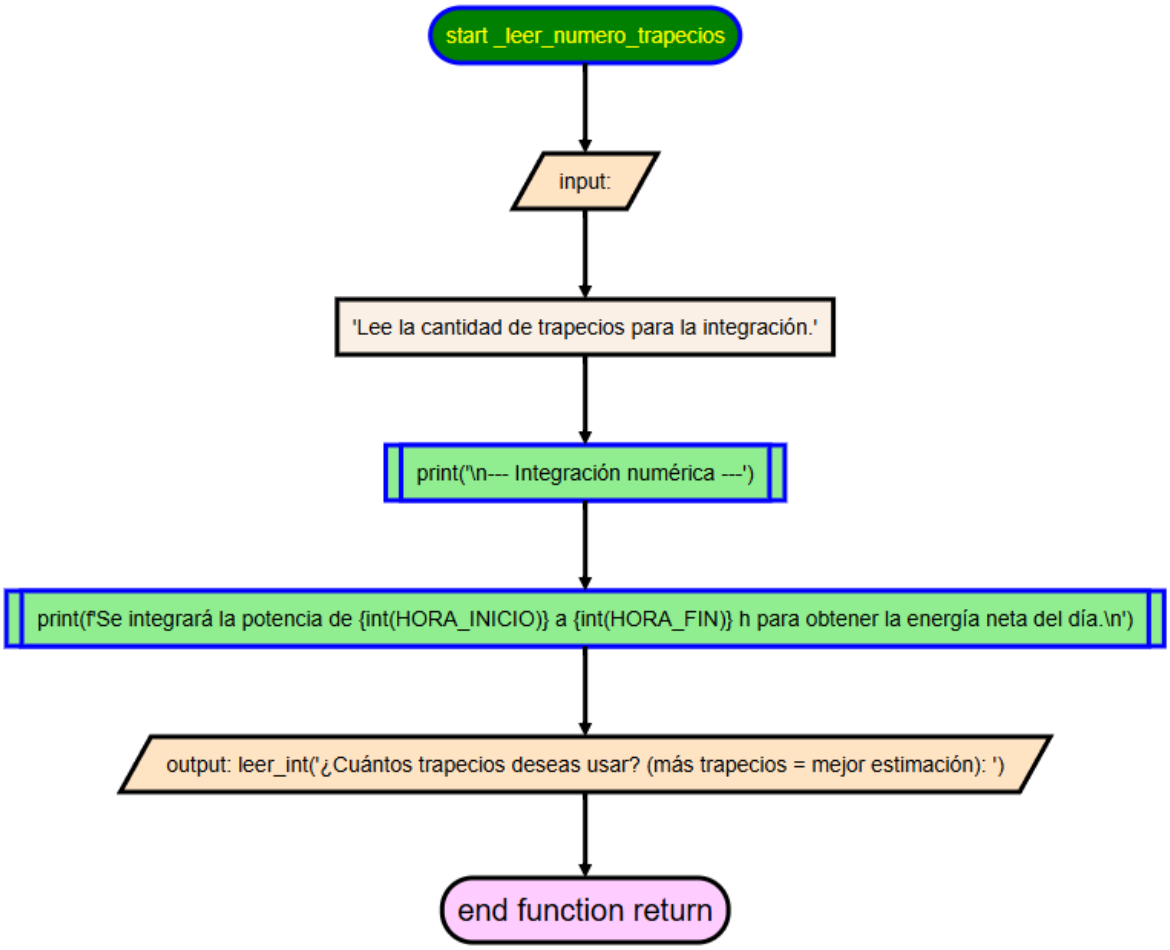
Mostrar Matriz



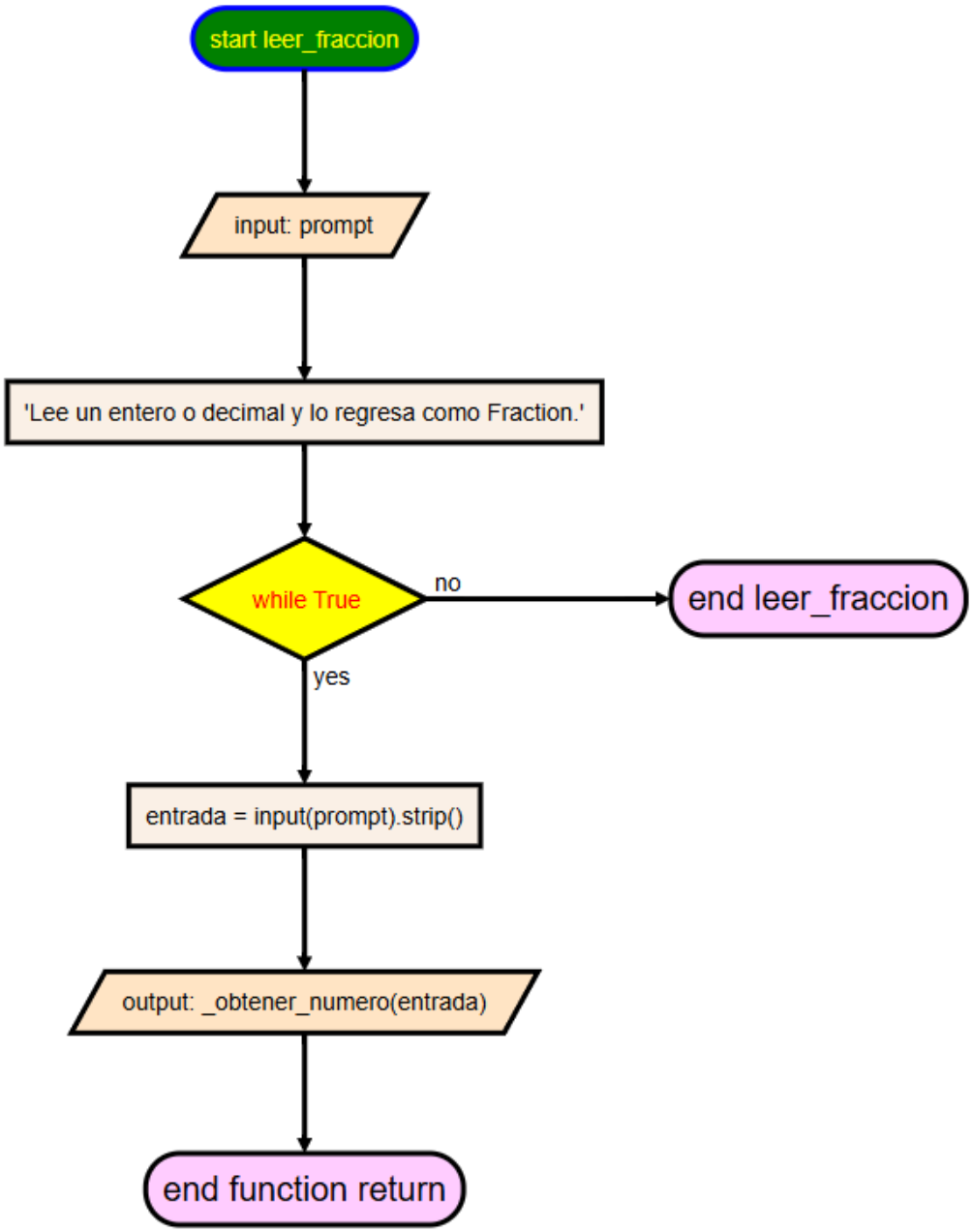
Leer Opción


start leer_opcion
input: prompt, lower, upper
'Lee un entero dentro del rango cerrado [lower, upper].'
while True
no
end leer_opcion
yes
entrada = input(prompt).strip()
valor = int(entrada)
if lower <= valor <= upper:
return valor
print(f'Opción fuera de rango (se esperaba entre {lower} y {upper})')
exception raised?
no
yes
except ValueError
no
yes
print(f'"{entrada}" no es un número entero válido.')

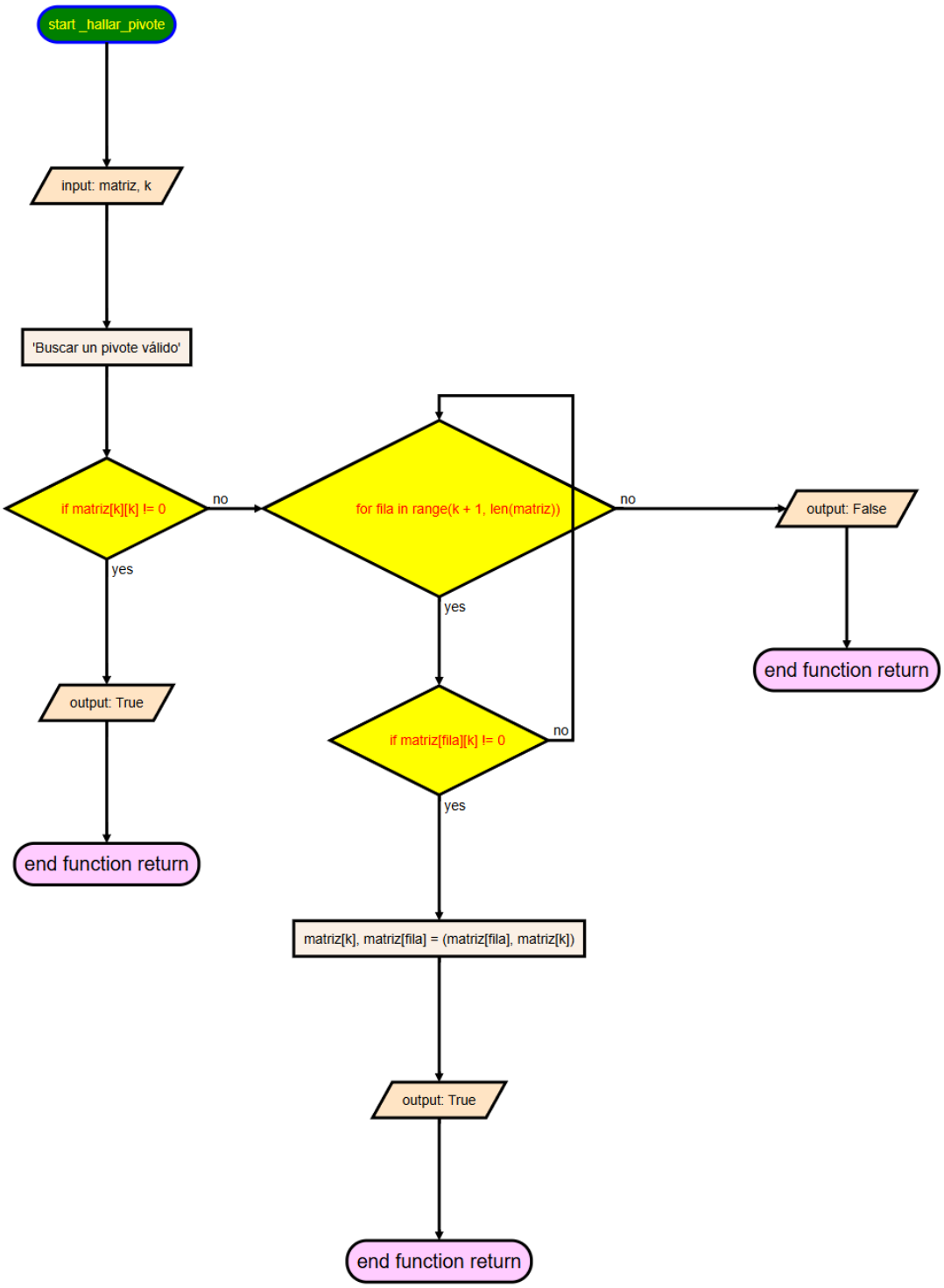
Leer Numero de Trapecios



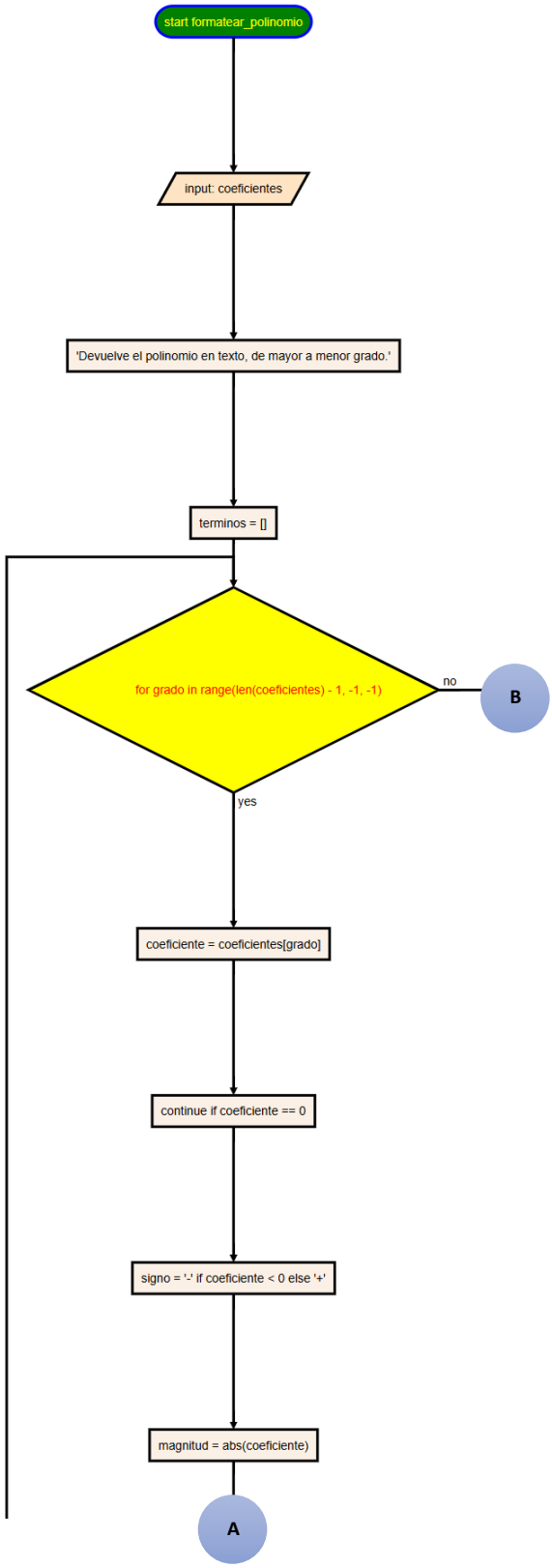
Leer Fracción

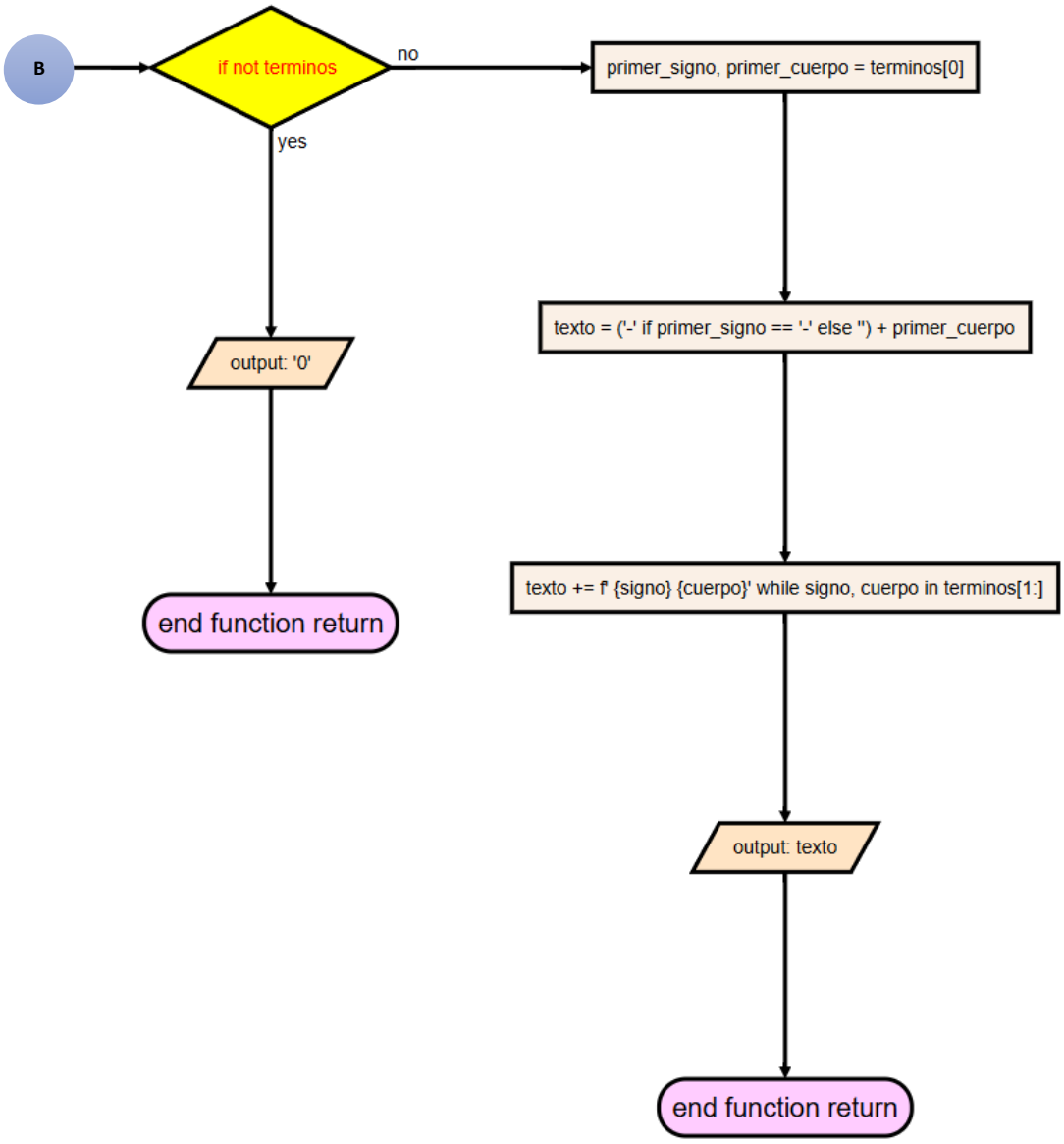


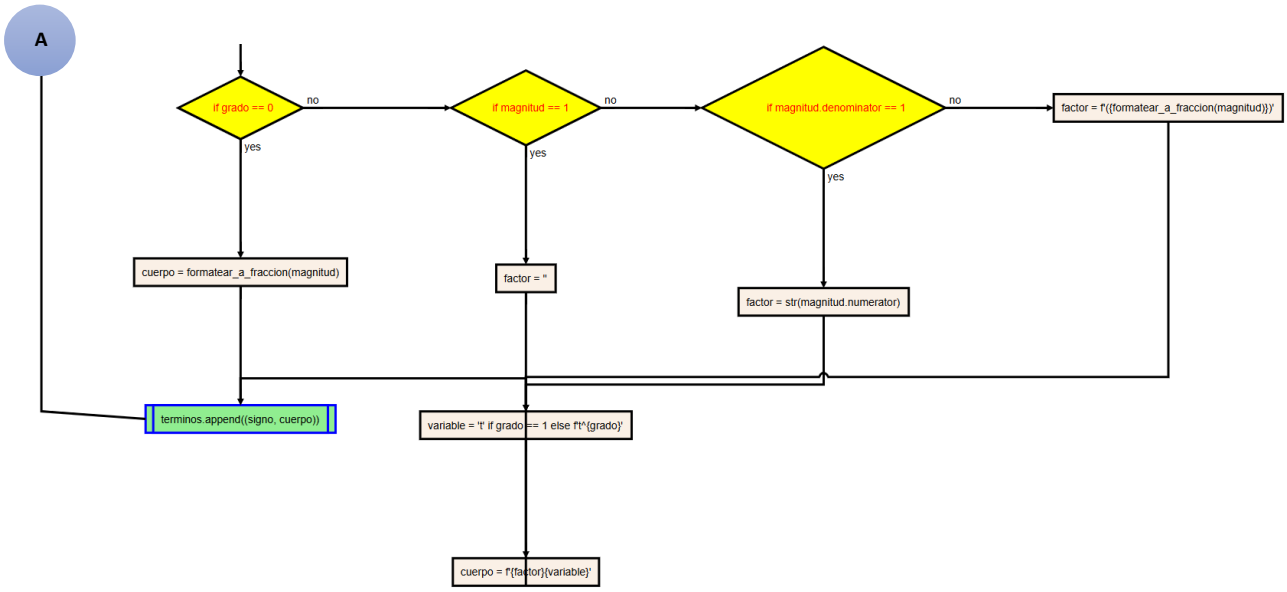
Hallar Pivote



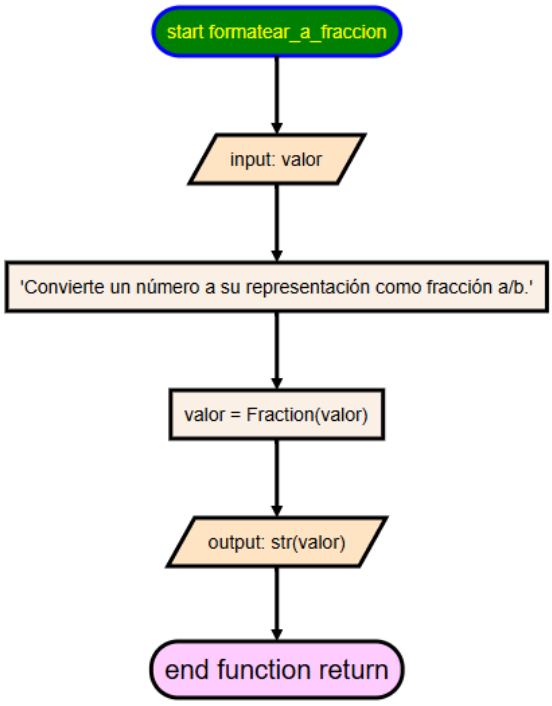
Formatear Polinomio



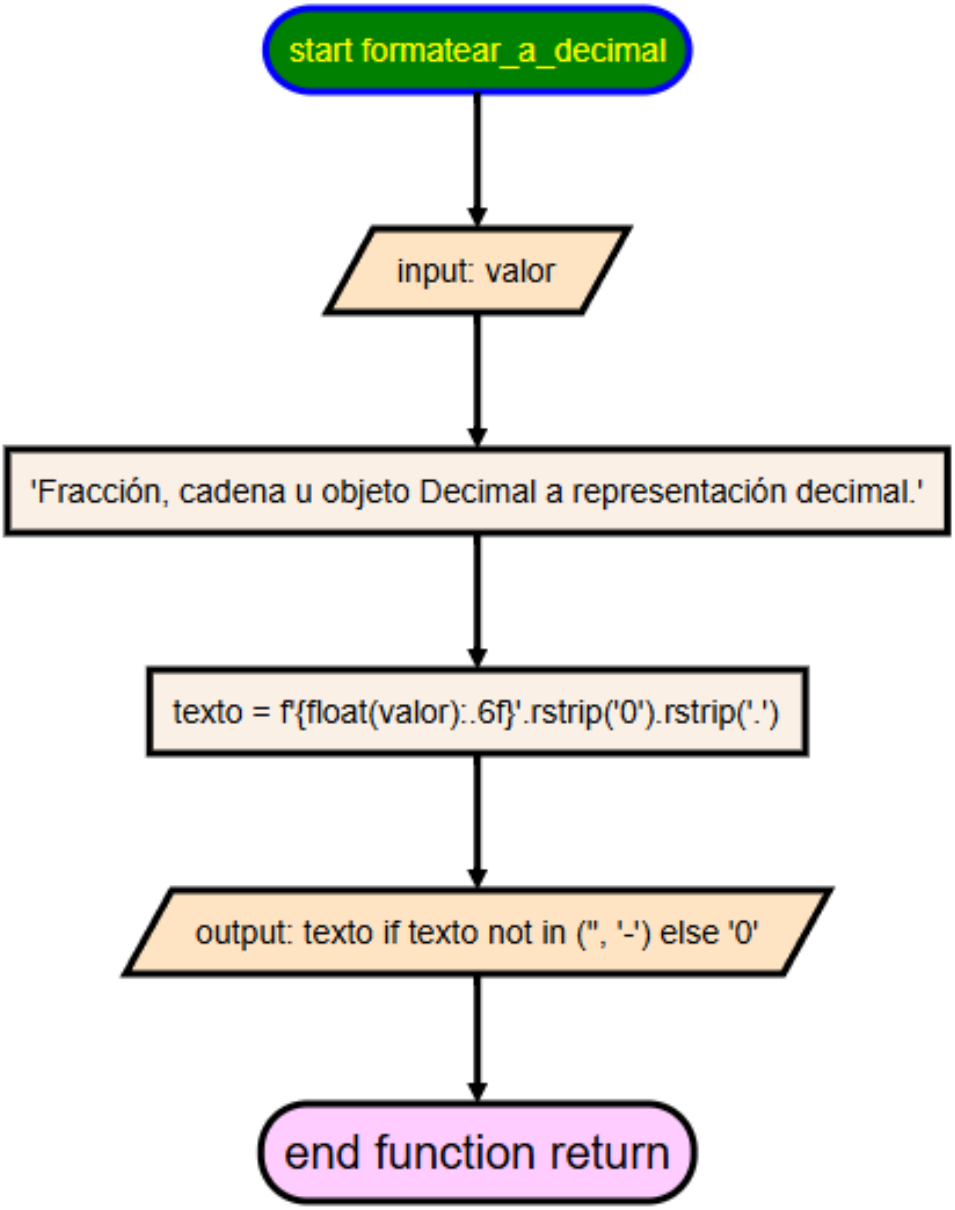

B
if not terminos
no
yes
primer_signo, primer_cuerpo = terminos[0]
texto = ('-' if primer_signo == '-' else '') + primer_cuerpo
texto += f' {signo} {cuerpo}' while signo, cuerpo in terminos[1:]
output: '0'
output: texto
end function return
end function return



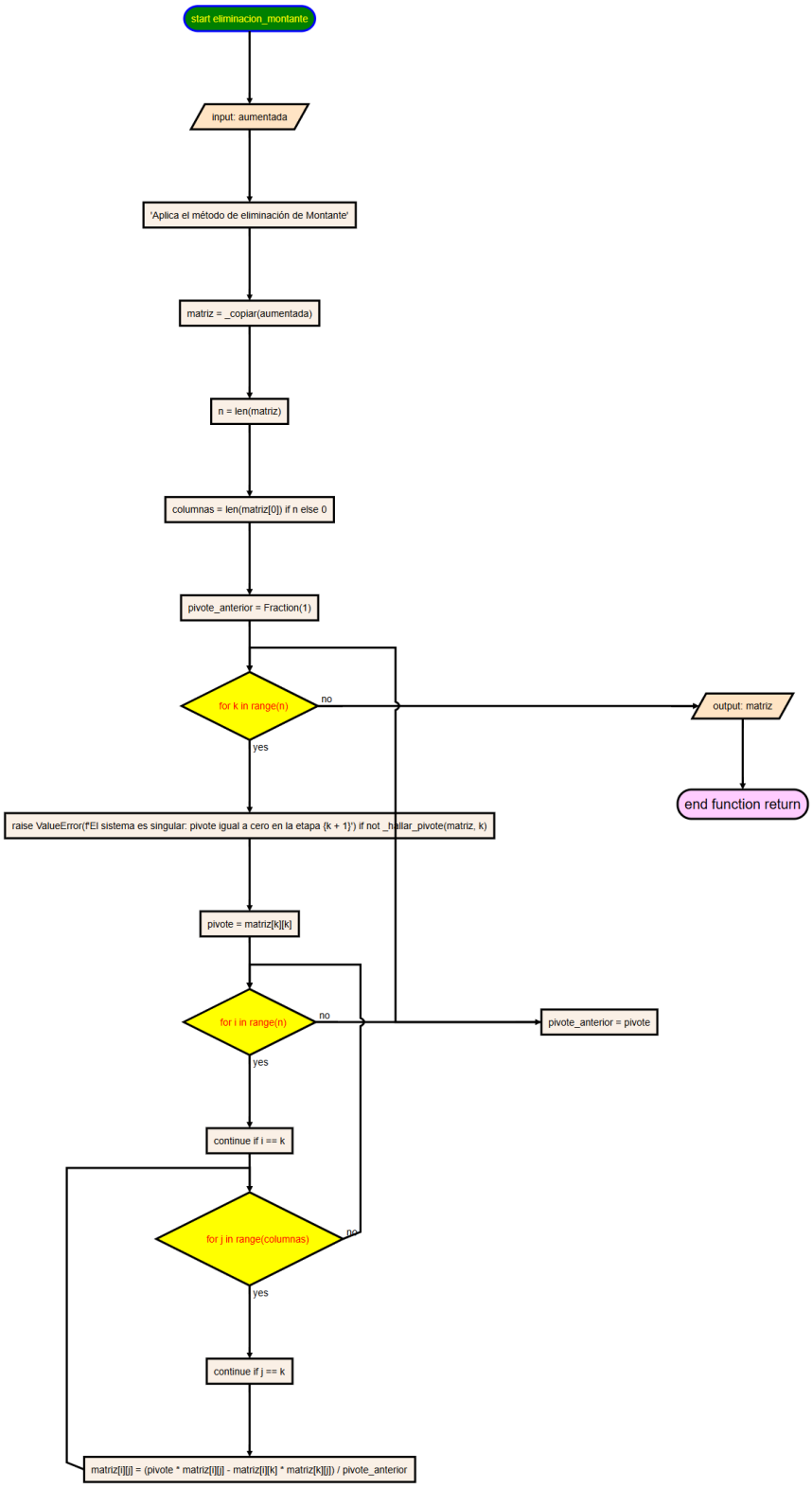
Formatear Fracción



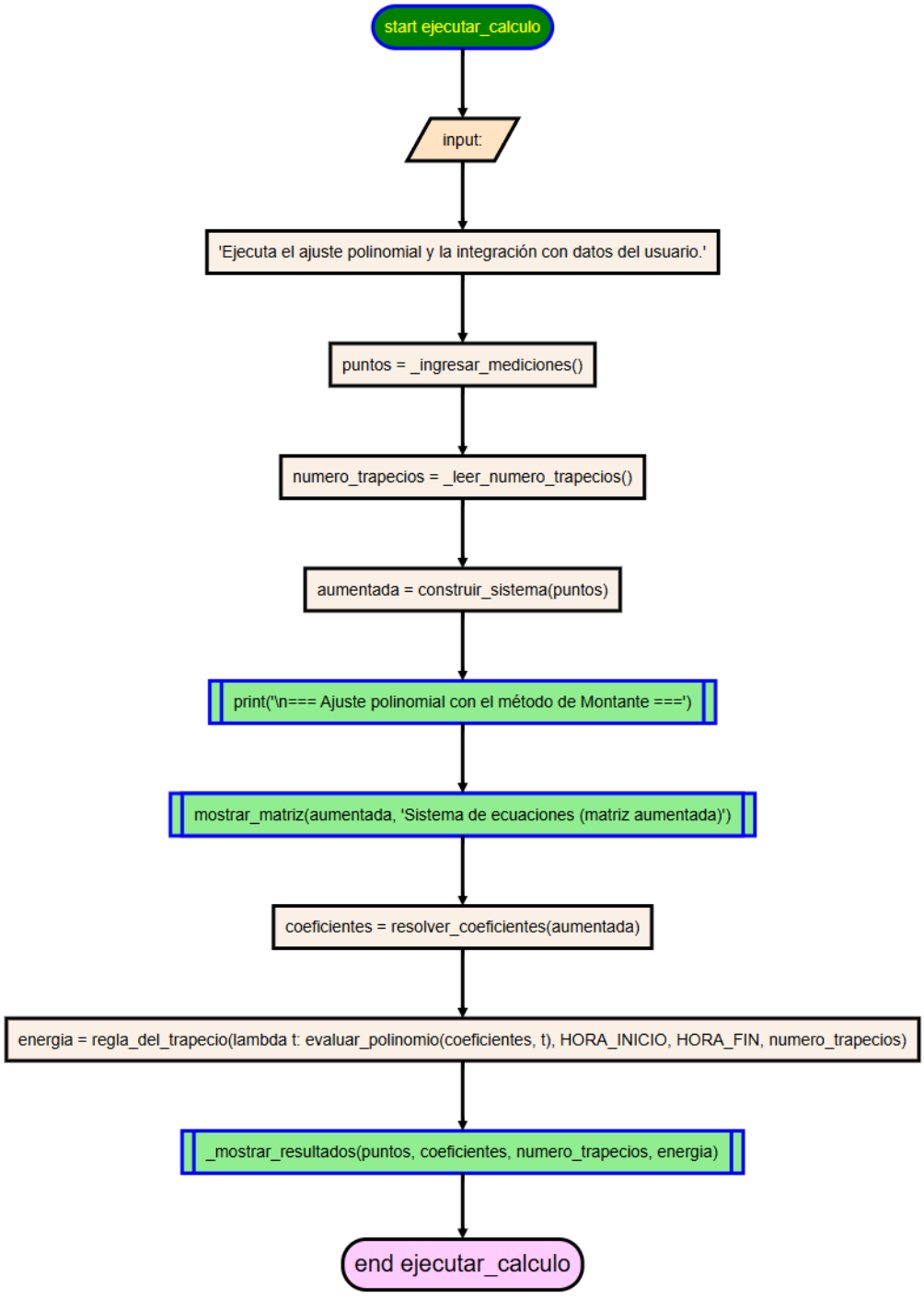
Formatear Decimal



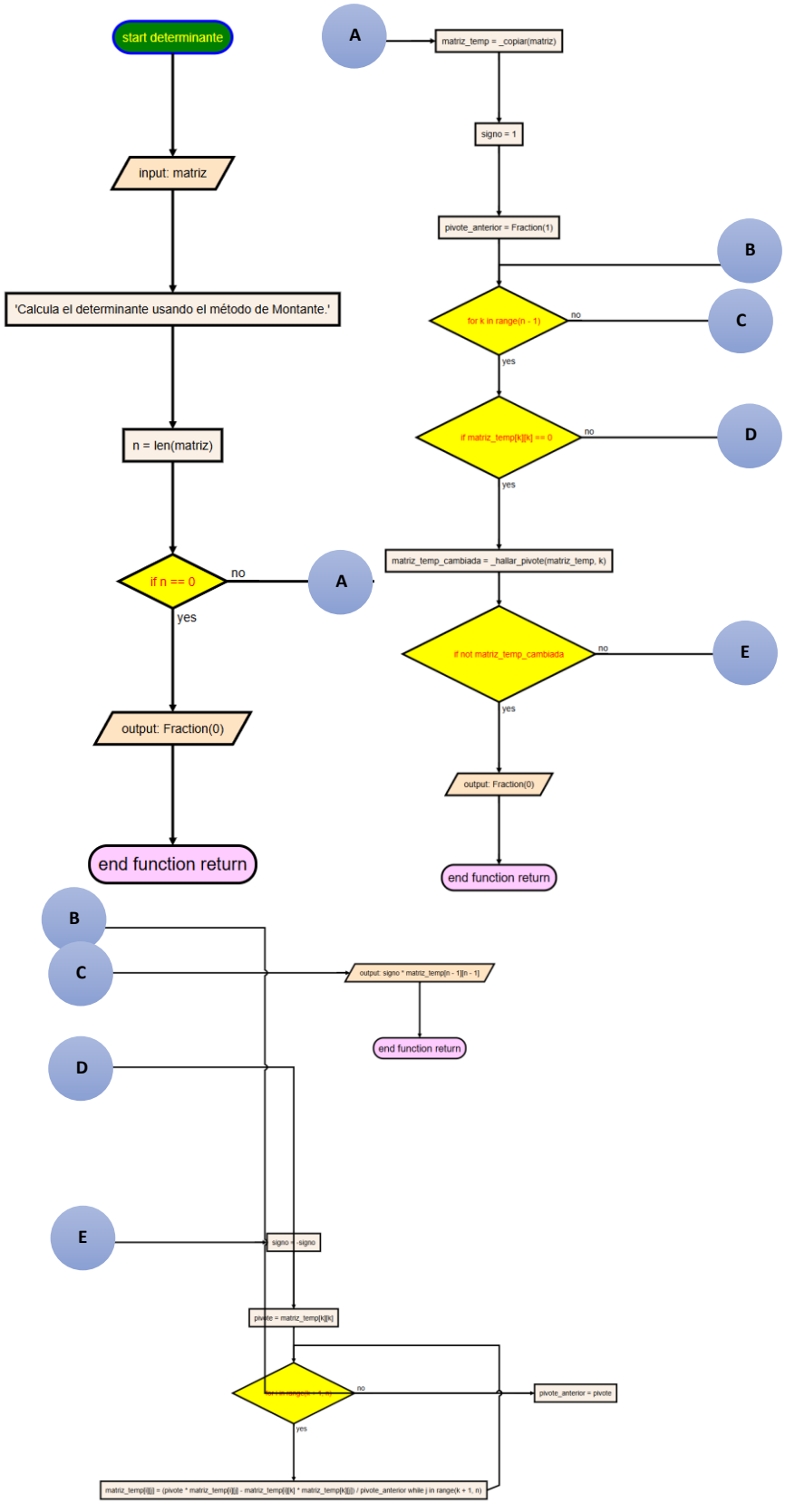
Eliminación Montante



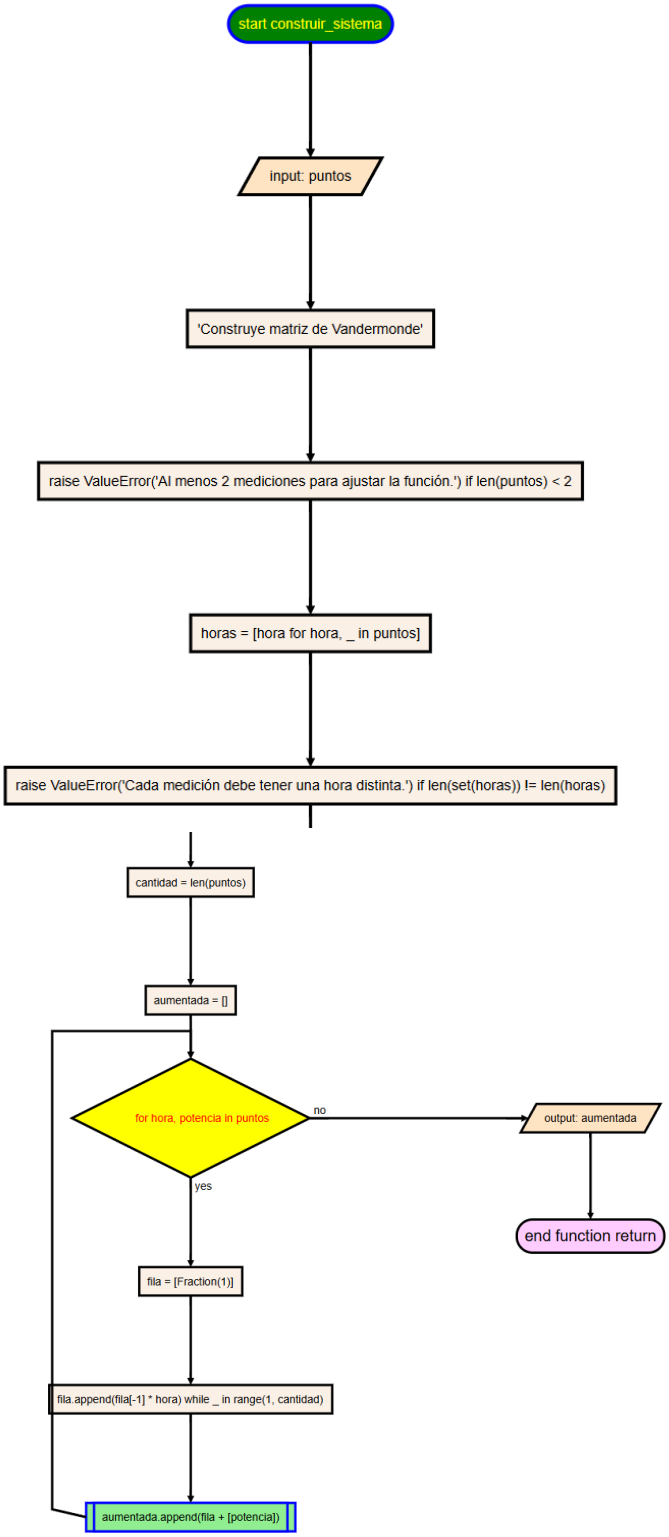
Ejecutar Cálculo



Determinante



Construcción del Sistema



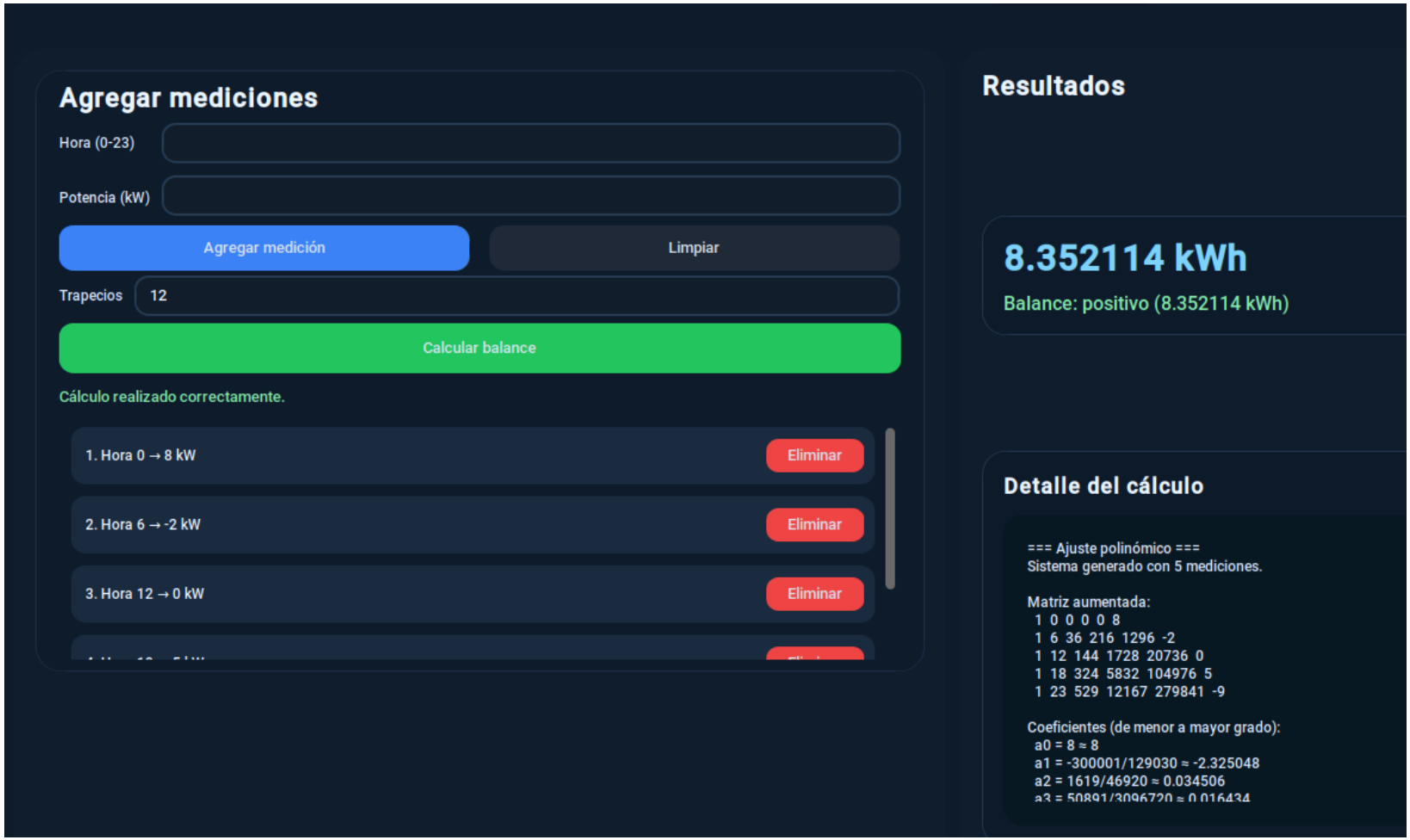
Conclusión general

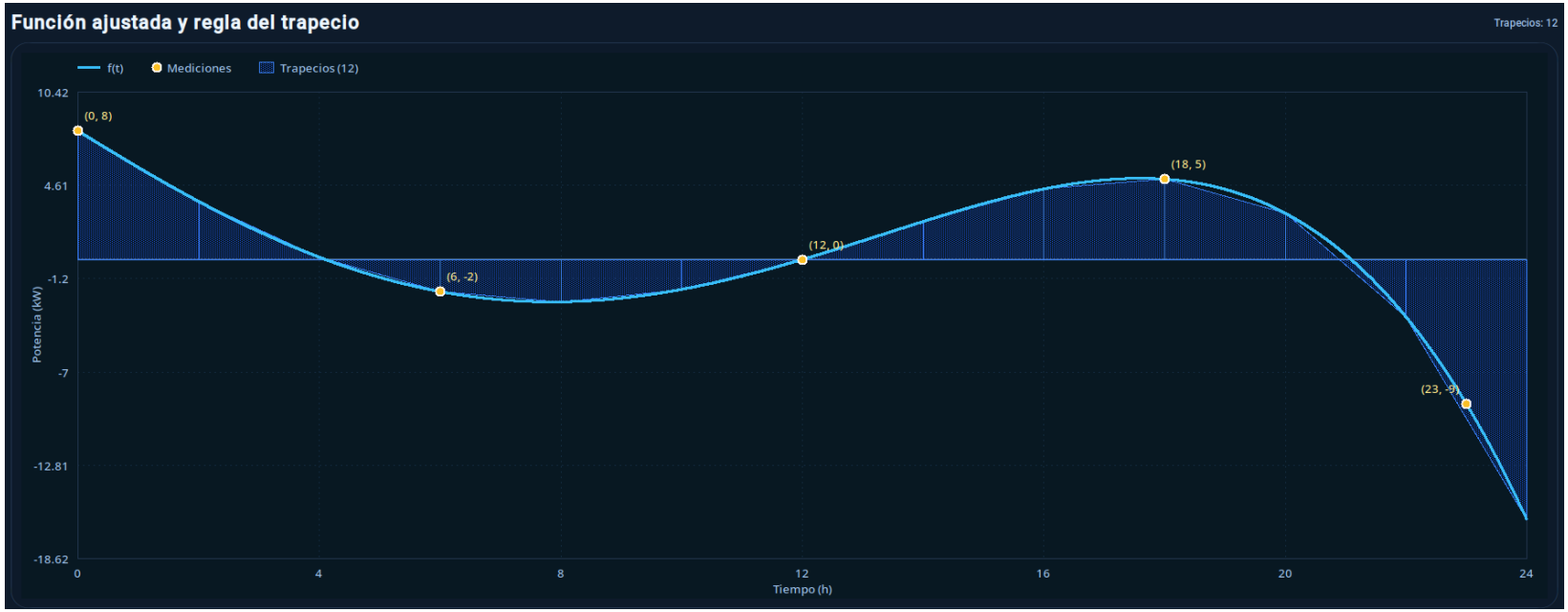
Con el desarrollo de este proyecto se aplicaron métodos numéricos para resolver un problema relacionado con el funcionamiento de las baterías de respaldo de un centro de datos. A partir de mediciones de potencia en diferentes horas, se construyó un polinomio de interpolación mediante el sistema de Vandermonde y se resolvieron sus coeficientes utilizando el método de Montante.

Posteriormente, se utilizó la regla compuesta del trapecio para aproximar la integral de la función de potencia durante un periodo de 24 horas. Esta integral permite estimar el cambio neto de energía: un resultado positivo indica que predominó la carga de las baterías, mientras que uno negativo indica que predominó la descarga.

Además de aplicar los procedimientos matemáticos, el proyecto permitió organizar el programa en módulos, validar los datos ingresados y presentar los resultados de manera comprensible. Se concluye que los métodos implementados son útiles para obtener una estimación del balance energético a partir de mediciones discretas. Sin embargo, el resultado depende de los datos proporcionados y del polinomio interpolador; cuando las mediciones no cubren todo el día, el modelo se extrapola a las horas restantes, por lo que la estimación debe interpretarse considerando esa limitación.

Ejemplo de ejecución




Función ajustada y regla del trapecio
Trapecios: 12
f(t)
Mediciones
Trapecios (12)
10.42
4.61
-1.2
-7
-12.81
-18.62
Potencia (kW)
(0, 8)
(6, -2)
(12, 0)
(18, 5)
(23, -9)
0
4
8
12
16
20
24
Tiempo (h)

Presentacion



Problematica

Los centros de datos cuentan con un banco de **baterías de respaldo (UPS)** que almacenan energía o la entregan según la demanda.

En la realidad, se tienen medidas discretas de potencias tomadas por sensores a ciertas horas del dia

Potencia positiva(+): Las baterias se cargaron con el excedente de energia
Potencia negativa(-): Se descargaron las baterias para mantener al servidor.

Nosotros calculamos en que medida se presenta una **Potencia positiva o negativa** en base a 2 o mas metricas discretas del comportamiento a las diversas horas.

Eleccion de los metodos

Integracion con Trapecios

La aproximación geométrica por trapecios procesa de forma nativa los cambios de signo de la función f(t). Las áreas por encima del **eje x (carga)** suman y las **áreas por debajo (descarga)** restan de forma automática, entregando el cambio neto en kWh sin perder exactitud en los subintervalos Δt

Metodo Montante

Para ajustar los datos discretos de los sensores a una función polinómica continua f(t) mediante una **matriz de Vandermonde.**

Problematica

Los centros de datos cuentan con un banco de **baterías de respaldo (UPS)** que almacenan energía o la entregan según la demanda.

En la realidad, se tienen medidas discretas de potencias tomadas por sensores a ciertas horas del dia

Potencia positiva(+): Las baterias se cargaron con el excedente de energia
Potencia negativa(-): Se descargaron las baterias para mantener al servidor.

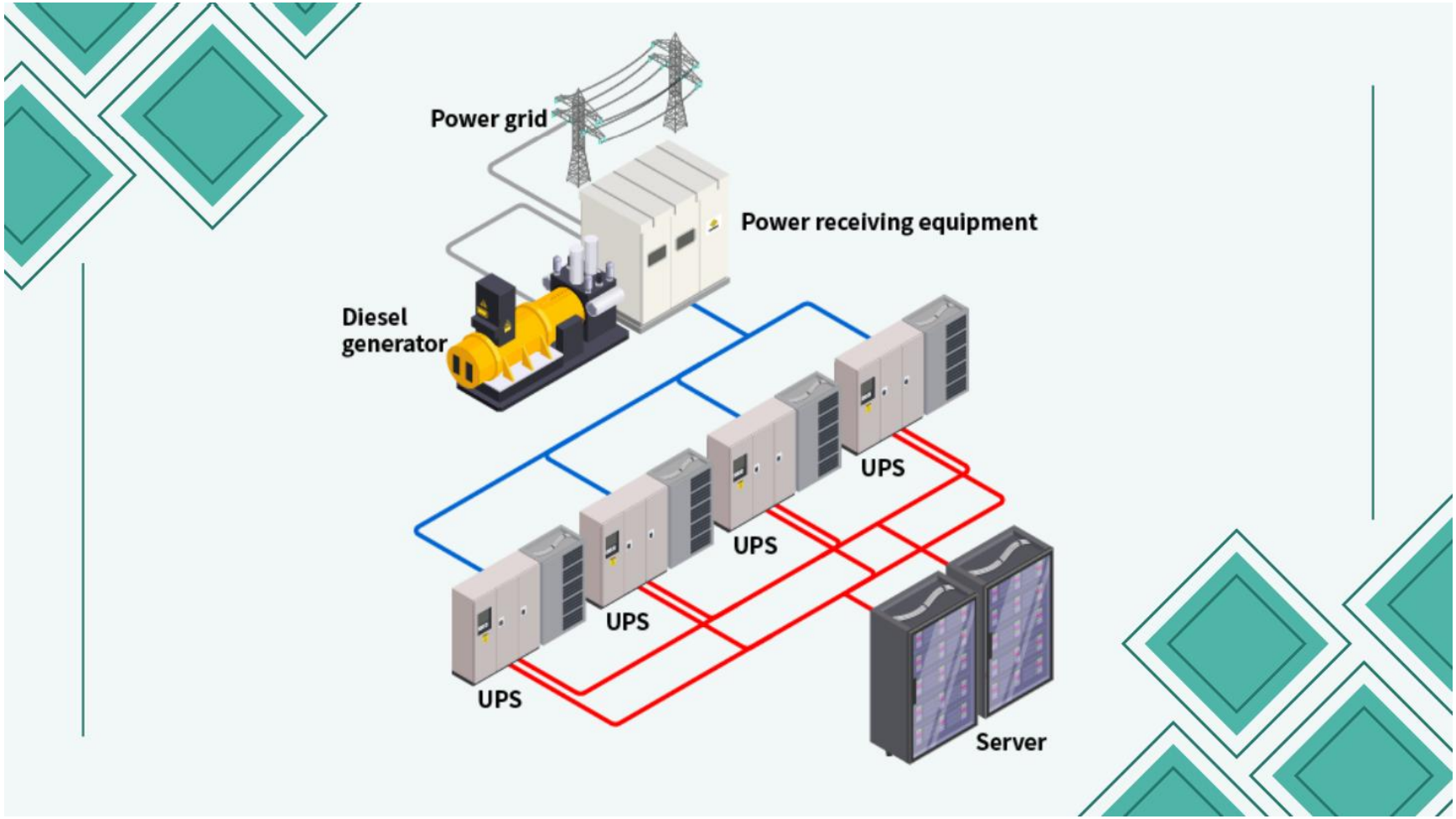
Nosotros calculamos en que medida se presenta una **Potencia positiva o negativa** en base a 2 o mas metricas discretas del comportamiento a las diversas horas.

Coulomb Counting

$$SOC(t) = \frac{1}{C}\int_0^t I(p)\,dp$$

total capacity

current


Power grid
Power receiving equipment
Diesel generator
UPS
UPS
UPS
UPS
Server



Ejemplo